COMP 372/471 Lecture Notes

Release 1.0

Konstantin Läufer

Dec 05, 2017

CONTENTS

1 Overview **3**
- 1.1 Motivation . . . 3
- 1.2 Key learning outcomes . . . 3
- 1.3 Some starting points for functional programming . . . 3
- 1.4 Course outline . . . 4
- 1.5 Bloom's taxonomy . . . 5

2 Context and Background **7**
- 2.1 Overview of a lightweight development process . . . 7
- 2.2 Software requirements . . . 8
- 2.3 Software design principles and patterns . . . 8
- 2.4 Programming language history and paradigms . . . 9
- 2.5 Popularity indices and performance comparisons . . . 10

3 The Imperative and Object-Oriented Paradigms in Scala **11**
- 3.1 Options for running Scala code . . . 11
- 3.2 The role of console applications . . . 12
- 3.3 Choices for testing Scala code . . . 14
- 3.4 Defining domain models in imperative and object-oriented languages . . . 15
- 3.5 Using Scala traits for modularity and dependency injection . . . 16

4 The Functional Programming Paradigm **19**
- 4.1 Solving problems using built-in types and behaviors . . . 19
- 4.2 Defining algebraic data types . . . 21
- 4.3 Behaviors on algebraic data types . . . 22
- 4.4 A closer look at predefined behaviors on lists . . . 22
- 4.5 Separation of concerns at the type level . . . 23
- 4.6 Other useful abstractions . . . 26
- 4.7 References . . . 27

5 Program Representation and Interpretation **29**
- 5.1 Programming language toolchains . . . 29
- 5.2 Compiler front-end concepts and techniques . . . 30
- 5.3 Programming language semantics . . . 31
- 5.4 Case study: a simple imperative language . . . 31
- 5.5 Case study: a simple imperative language with records . . . 35
- 5.6 Type systems . . . 42
- 5.7 Domain-specific languages . . . 43

6 Introduction to Concurrency **47**
- 6.1 Overview . . . 47

6.2 Activity terminology and concerns . . . 47
6.3 Thread safety . . . 48
6.4 Dealing with shared state . . . 49
6.5 (Conflicting) design forces . . . 49
6.6 Specific concurrency mechanisms . . . 49
6.7 References: concurrent and asynchronous computing . . . 50

7 Summary **51**

8 Appendix: Programming Language Design Principles **53**

9 Appendix: Resources **55**
9.1 Key resources . . . 55
9.2 Code examples . . . 55
9.3 Required text . . . 56
9.4 Recommended/supplementary texts . . . 56
9.5 Additional texts on Scala . . . 56
9.6 Online Scala resources . . . 56
9.7 Resources on program representation and interpretation . . . 57
9.8 Other resources . . . 57

10 Appendix: Course Software **59**
10.1 Cloud-based development environment . . . 59
10.2 Locally installed development environment . . . 60
10.3 sbt optional plugins . . . 61
10.4 Bitbucket . . . 61
10.5 GitHub (optional/recommended) . . . 62
10.6 Remote participation . . . 62

11 Appendix: Course Syllabus **63**
11.1 Course: COMP 372/471 Programming Languages . . . 63
11.2 Section: 001 Spring 2018 . . . 63
11.3 Section: 001 Spring 2017 . . . 63
11.4 Detailed Course Outline . . . 64
11.5 Important Dates . . . 65
11.6 Key Resources . . . 65

12 Appendix: TODO **67**

COMP 372 | COMP 471
Department of Computer Science
Loyola University Chicago

Welcome! In this course, we study programming language concepts and paradigms in the context of contemporary software engineering practice. This version of the course is normally taught by Konstantin Läufer and focuses on functional and concurrent programming languages with examples and projects mostly in Scala.

We maintain these lecture notes using GitHub. We also offer PDF and ePub formats for offline/mobile reading.

Warning: These notes are still being written, so expect a few rough edges. But we're getting closer! Your contributions are *highly welcome!* Please submit issues and pull requests.

CHAPTER

ONE

OVERVIEW

In this chapter, we provide an outline of the course, along with a brief motivation and key learning outcomes.

1.1 Motivation

There are a number of reasons why you would want to study programming languages. To name a few:

- Knowing several languages will help you solve more problems in better ways.
- The subject matter is exciting and intellectually challenging.
- Google, Meetup, and increasing numbers of local startups love it (from an employer's point of view)!

1.2 Key learning outcomes

We deliberately chose the learning outcomes for this course to complement other courses in your computer science or software engineering curriculum. Upon entering this course, you will already be familiar with the imperative and object-oriented paradigms, coming from languages such as Java, C++, C#, or Python. Most likely, you will also be familiar with basic machine architecture and memory management using C or C++.

In this course, we will pursue one overarching learning outcome and three more specific ones:

- an understanding of the programming language design space
- proficiency in functional programming
- an understanding of program representation and interpretation/execution
- basic competence in concurrent programming

Given the widespread availability in multi-core hardware ranging from mobile devices to enterprise servers, there is a growing interest in programming paradigms and techniques that will take advantage of this hardware. Functional programming, which de-emphasizes mutable state, in conjunction with suitable concurrency constructs, is particularly useful. We will also "look under the hood" by studying how to represent programs and interpret, execute, or otherwise process those representations.

1.3 Some starting points for functional programming

You might find the following resources useful to get into the functional programming mindset:

- Presentation by Martin Odersky: Scala with Style (YouTube)

- Interview with Simon Peyton-Jones on Functional Programming and Haskell (SE Radio audio podcast)
- Presentation by Läufer, Thiruvathukal, and Kaylor: The Promise of Statically Typed Functional Programming: Broader Context and Language Comparison (visual presentation)

Todo

incorporate visual presentation here

1.4 Course outline

Here is a more detailed course outline (subject to revision).

- functional programming (6 weeks)
  - defining algebraic data types
    - scalars: enumerations
    - sublinear structures: numbers, option
    - linear structures: lists, maps
    - nonlinear structures: trees
  - implementing behaviors on algebraic data types
    - pattern matching
    - recursion
  - higher-order functions
  - predefined types and their behaviors
  - recursion patterns
  - higher-kinded types
- programming language representation and interpretation/execution (5 weeks)
  - language design space
  - scanners (lexical analyzers)
  - parsers (syntax analyzers)
  - interpreters and compilers
  - domain-specific languages
- concurrent and parallel programming (4 weeks)
  - parallel collections
  - futures and promises
  - progress reporting and cancelation
  - asynchronous programming/reactive extensions (Rx)
  - advanced mechanisms
    - explicit threads

    * actors
    * software-transactional memory
    * task-parallel library
    * functional data structures

1.5 Bloom's taxonomy

The letters refer to the successive levels of learning from the cognitive domain of Bloom's taxonomy:

- K: know/remember the term
- C: comprehend/understand the concept
- A: apply the technique
- analyze/evaluate/create (advanced levels)

Todo

map Bloom levels to course outline

CHAPTER

TWO

CONTEXT AND BACKGROUND

In this chapter, we establish a practical context and provide some background information for the study of programming languages.

2.1 Overview of a lightweight development process

A successful development process usually comprises these minimal elements:

- automated regression testing
  - tests represent expectations of how the software should behave
  - when expressed as code, these are
    - fun to produce (like other coding)
    - convenient to run frequently
  - fix subject-under-test (SUT) (not tests) until tests pass
  - retest every time
    - a feature is added
    - the code is refactored
- refactoring
  - improve the quality of the code without changing its behavior
    - macro level: nonfunctional requirements (quality factors)
    - micro level: code smells
  - catalog of refactorings
- continuous integration

The process tree example illustrates continuous integration using various hosted services:

- Travis CI: continuous integration
- Codecov: test coverage
- Codacy: automated code review
- IssueStats and IsItMaintained: stats on issues and pull requests

The click counter example includes additional hosted continuous integration and delivery targets suitable for mobile app development.

2.2 Software requirements

In most cases, we develop software to provide some form of value:

- learn a language, library, framework, platform, technique, or tool (see also the ThoughtWorks Technology Radar)
- solve a problem
- produce an asset

There is usually some tension among these three activities.

The basic categories of requirements are

- functional (FR)
  - output as function of input: $y = f(x)$
  - or some other description of observable behavior
    - batch
    - interactive/event-based
- nonfunctional (NFR): additional properties of f, e.g.
  - testability
    - most important nonfunctional requirement
    - allows testing whether functional requirements are met
    - good architecture often happens as a side-effect (APPP pp. 36-38), such as separating I/O from core functionality
  - performance
  - scalability
    - e.g. performance for large data sets: asymptotic order of complexity
    - (big-Oh) in terms of input size n
  - reliability
  - maintainability
  - static versus dynamic NFRs

Several common questions and issues related to requirements arise:

- *how do requirements relate to the project development lifecycle?*
- *BUFD versus MVP*
- *how do testing and refactoring relate to requirements?*

The following figure by Kazman relates unit operations (high-level generalizations of refactorings) and software quality factors (nonfunctional requirements).

2.3 Software design principles and patterns

The software development community has identified various principles intended to guide the design and development process, for example:

Table 2: Quality Factors Versus Design Operations

Unit Operation	Software Quality Factor							
	Scalability	Modifiability	Integrability	Portability	Performance	Reliability	Ease of Creation	Reusability
Separation	+	+	+	+	+/-		+/-	+
Abstraction	+	+	+	+	-		+	+
Compression	-	-	-	-	+		+/-	-
Uniform Composition	+		+				+	
Replication	-	-		-	+/-	+	-	-
Resource Sharing		+	+	+	+/-	-	+	+/-

- DRY (don't repeat yourself)
- SoC (separation of concerns)
- SOLID

The community has also developed a body of design patterns that represent reusable solutions to recurring problems. Some key design patterns we will rely on in this course include

- Iterator
- Strategy
- Composite
- Decorator
- Visitor
- Abstract Factory
- Observer

We will recap these throughout the course as needed.

Note: Language-specific design patterns are called *idioms*.

2.4 Programming language history and paradigms

The resources in this section cover fundamental models of computation, language paradigms, and language principles.

- overview talk by Läufer and Thiruvathukal
- programming languages paradigms: diagram by Van Roy
- programming languages paradigms: book chapter by Van Roy

- *Appendix: Programming Language Design Principles* by MacLennan
- Turing completeness
- Church-Turing thesis

Todo

expand into a proper section

2.5 Popularity indices and performance comparisons

There are a number of programming language popularity indices and performance comparisons. Before drawing any conclusions from these indices, it is important to understand their *methodology*.

- PYPL PopularitY of Programming Language index
- TIOBE programming community index
- GitHub language popularity
- Programming languages shootout benchmark

CHAPTER

THREE

THE IMPERATIVE AND OBJECT-ORIENTED PARADIGMS IN SCALA

In this chapter, we discuss the imperative and object-oriented programming paradigms with examples in Scala.

3.1 Options for running Scala code

In this section, we discuss the different options for running Scala code, including applications and tests.

- The simplest way to run Scala code fragments is through the Scala REPL (read-eval-print loop). We can launch the Scala REPL and then evaluate definitions and expressions:

```
$ scala
Welcome to Scala 2.12.1 (Java HotSpot(TM) 64-Bit Server VM, Java 1.8.0_102).
Type in expressions for evaluation. Or try :help.

scala> 3 + 4
res0: Int = 7

scala> def f(x: Int) = x + 2
f: (x: Int)Int

scala> f(3)
res1: Int = 5

scala> val z = f(4)
z: Int = 6

scala> Seq(1, 2, 3).map(f)
res2: Seq[Int] = List(3, 4, 5)
```

  This is a very effective, painless way to conduct initial explorations. The drawback of this approach is a lack of support for *managed dependencies*, which are required for more advanced work. In that case, starting the Scala REPL through sbt as discussed below is a much better choice. Managing the Scala/Java classpath manually is discouraged.

  You can also run simple scripts (with optional command-line arguments) directly through the scala interpreter:

```
$ cat > blah.scala
println(args.toList)
$ scala blah.scala 1 2 3
List(1, 2, 3)
```

- In a Scala IDE such as IntelliJ IDEA, we can run Scala applications (classes/objects with a `main` method or objects extending the `App` trait) and Scala tests from within the IDE. To pass command-line arguments to an application, we have to create a suitable run configuration.

- In IntelliJ IDEA, Scala worksheets are a convenient way to do REPL-style exploratory programming while being able to modify and save all your code. Check out this example, especially the simple option and list examples. For simple testing, you can intersperse assertions within your code. Worksheets can live in any folder and coexist with other code. So you can start exploring something in a worksheet and then move it into your production code when appropriate. The Eclipse Scala IDE has a similar feature.
- It is best to use sbt (the Scala Build Tool) for projects with one or more external dependencies because of sbt's (and similar build tools') ability to manage these dependencies in a declarative way:

```
$ sbt test
$ sbt run
$ sbt "run arg1 arg2 ..."
$ sbt "runMain my.pkg.Main arg1 arg2 ..."
$ sbt test:run
```

  In addition, sbt allows you to start a REPL that exposes the code in your project and its managed dependencies. This is the preferred way to explore existing libraries:

```
$ sbt console
```

  You can also pull in the additional dependencies from the test scope:

```
$ sbt test:console
```

  If you want to bypass your own code in case of, say, compile-time errors, you can use one of these tasks:

```
$ sbt consoleQuick
$ sbt test:consoleQuick
```

  In conjunction with a text editor, sbt's triggered execution for testing will significantly shorten the edit-compile-run/test cycle, for example:

```
$ sbt
...
> ~ test
```

- Finally, to turn an sbt-based Scala application into a script you can run outside sbt, you can use the sbt-native-packager plugin. To use this plugin, add this line to the end of `build.sbt`:

```
enablePlugins(JavaAppPackaging)
```

  and this one to `project/plugins.sbt`:

```
addSbtPlugin("com.typesafe.sbt" % "sbt-native-packager" % "1.1.4")
```

  Then, after any change to your sources, you can create/update the script and run it from the command line like so:

```
$ sbt stage
...
$ ./target/universal/stage/bin/myapp-scala arg1 arg2 ...
```

3.2 The role of console applications

Console applications have always been an important part of the Unix command-line environment. The typical console application interacts with its environment in the following ways:

- zero or more application-specific *command-line arguments* for passing options to the application: `app arg1 arg2 ...`
- *standard input* (stdin) for reading the input data
- *standard output* (stdout) for writing the output data
- *standard error* (stderr) for displaying error messages separately from the output data

Applications written in this way can function as composable building blocks using Unix pipes. Using these standard I/O mechanisms is much more flexible than reading from or writing to files whose names are hardcoded in the program.

E.g., the `yes` command outputs its arguments forever on consecutive output lines, the `head` command outputs a finite prefix of its input, and the `wc` command counts the number of characters, words, or lines:

```
yes hello | head -n 10 | wc -l
```

You may wonder how the upstream (left) stages in the pipeline know when to terminate. Concretely, how does the `yes` command know to terminate after `head` reads the first ten lines. When `head` is done, it closes its input stream, and `yes` will receive an error signal called `SIGPIPE` when it tries to write further data to that stream. The default response to this error signal is termination.

We can also use the control structures built into the shell. E.g., the following loop prints an infinite sequence of consecutive integers starting from 0:

```
n=0 ; while :; do echo $n ; ((n=n+1)) ; done
```

These techniques are useful for producing test data for our own applications. To this end, we can redirect output to a newly created file using this syntax:

```
n=0 ; while :; do echo $n ; ((n=n+1)) ; done > testdata.txt
```

If `testdata.txt` already exists, it will be overwritten when using this syntax. We can also append to an existing file:

```
... >> testdata.txt
```

Similarly, we can redirect input from a file using this notation:

```
wc -l < testdata.txt
```

There is a close relationship between Unix pipes and functional programming: When viewing a console application as a function that transforms its input to its output, Unix pipes correspond to function composition. The pipeline `p | q` corresponds to the function composition `q o p`.

3.2.1 Console applications in Scala

The following techniques are useful for creating console applications in Scala. As in Java, command-line arguments are available to a Scala application as `args` of type `Array[String]`.

We can read the standard input as lines using this iterator:

```
val lines = scala.io.Source.stdin.getLines
```

This gives you an iterator of strings with each item representing one line. When the iterator has no more items, you are done reading all the input. (See also this concise reference.)

To break the standard input down further into words, we can use this recipe:

```
val words = lines.flatMap(_.split("\\W+"))
```

In Scala, `print` and `println` print to stdout.

Console applications written in this way can be part of a Unix pipeline only to a limited extent. Unfortunately, the Java virtual machine ignores the `SIGPIPE` error signal, so we cannot use Scala (or Java) console applications as upstream components that produce an infinite output sequence and depend on this signal.

3.2.2 The importance of constant-space complexity

Common application scenarios involve large volumes of input data or infinite input streams, e.g., sensor data from an internet-of-things device. To achieve reliability and scalability of such applications, it is critical to ensure that the application does not exceed a constant memory footprint during its execution.

Concretely, whenever possible, this means processing one input item at a time and then forgetting about it, rather than storing the entire input in memory. This version of a program that echoes back and counts its input lines has constant-space complexity:

```
var count = 0
for (line <- lines) {
  println(line)
  count += 1
}
println(line + " lines counted")
```

By contrast, this version has linear-space complexity and may run out of space on a large volume of input data:

```
var count = 0
val listOfLines = lines.toList
for (line <- listOfLines) {
  println(line)
  count += 1
}
println(line + " lines counted")
```

In sum, to achieve constant-space complexity, it is usually best to represent the input data as an iterator instead of converting it to an in-memory collection such as a list. Iterators support most of the same behaviors as in-memory collections.

3.3 Choices for testing Scala code

There are various basic techniques and libraries/frameworks for testing Scala code.

The simplest way is to intersperse assertions within your code. This is particularly effective for scripts and worksheets:

```
val l = List(1, 2, 3)
assert { l.contains(2) }
```

The following testing libraries/frameworks work well with Scala.

- The familiar JUnit can be used directly.
- ScalaCheck is a testing framework for Scala that emphasizes property-based testing, including universally quantified properties, such as "for all lists `x` and `y`, the `(x ++ y).length` is equal to `x.length + y.length`"

- ScalaTest is a testing framework for Scala that supports a broad range of test styles including behavior-driven design, including integration with ScalaCheck.
- specs2 is a specification-based testing library that also supports integration with ScalaCheck.

For faster turnaround, we can combine these techniques with triggered execution.

The echotest example shows some of these libraries in action.

3.4 Defining domain models in imperative and object-oriented languages

In imperative and object-oriented languages, the basic type abstractions are

- addressing: pointers, references
- aggregation: structs/records, arrays
  - example: node in a linked list
- variation: tagged unions, multiple implementations of an interface
  - example: mutable set abstraction
    - add element
    - remove element
    - check whether an element is present
    - check if empty
    - how many elements
  - several possible implementations
    - reasonable: binary search tree, hash table, bit vector (for small underlying domains)
    - less reasonable: array, linked list
    - see also this table of collection implementations

In an object-oriented language, we commonly use a combination of design patterns (based on these basic abstractions) to represent domain model structures and associated behaviors:

- https://github.com/lucoodevcourse/shapes-android-java
- https://github.com/LoyolaChicagoCode/misc-java/blob/master/src/main/java/expressions/SimpleExpressions.java
- https://github.com/LoyolaChicagoCode/misc-java/blob/master/src/main/java/vexpressions/VisitorExpressions.java

3.4.1 Object-oriented Scala as a "better Java"

Scala offers various improvements over Java, including:

- unified types
- standalone higher-order functions
- standalone objects

- case classes and pattern matching
- traits: generalization of interfaces and restricted form of abstract classes, can be combined/stacked
- package structure decoupled from folder hierarchy
- higher-kinded types (advanced topic)

More recent versions of Java, however, have started to echo some these advances:

- lambda expressions
- default methods in interfaces

We will study these features as we encounter them.

Todo

examples below after discussing the next topic

3.5 Using Scala traits for modularity and dependency injection

Scala traits are *abstract* types that can serve as fully abstract interfaces as well as partially implemented, composable building blocks (mixins). Unlike Java interfaces (prior to Java 8), Scala traits can have method implementations (and state). The Thin Cake idiom shows how traits can help us achieve the following two design goals:

- testability
- avoidance of code duplication ("DRY")

Note: We deliberately call *Thin Cake* an *idiom* as opposed to a pattern because it is *language-specific*.

We will draw examples for this section from the process tree example.

First, to achieve testability, we can define the desired functionality, such as `common.IO`, as its own trait instead of a concrete class or part of some other trait such as `common.Main`. Such traits are *providers* of some functionality, while building blocks that use this functionality are *clients*, such as''common.Main'' (on the production side) and `PrintSpec` (on the testing side). Specifically, we use `PrintSpec` to test `common.IO` in isolation, independently of `common.Main`.

To manage the growing complexity of a system, we usually try to decompose it into its design dimensions. In the process tree example, we recognize these design dimensions:

- mutable versus immutable implementation
- run in production versus testing

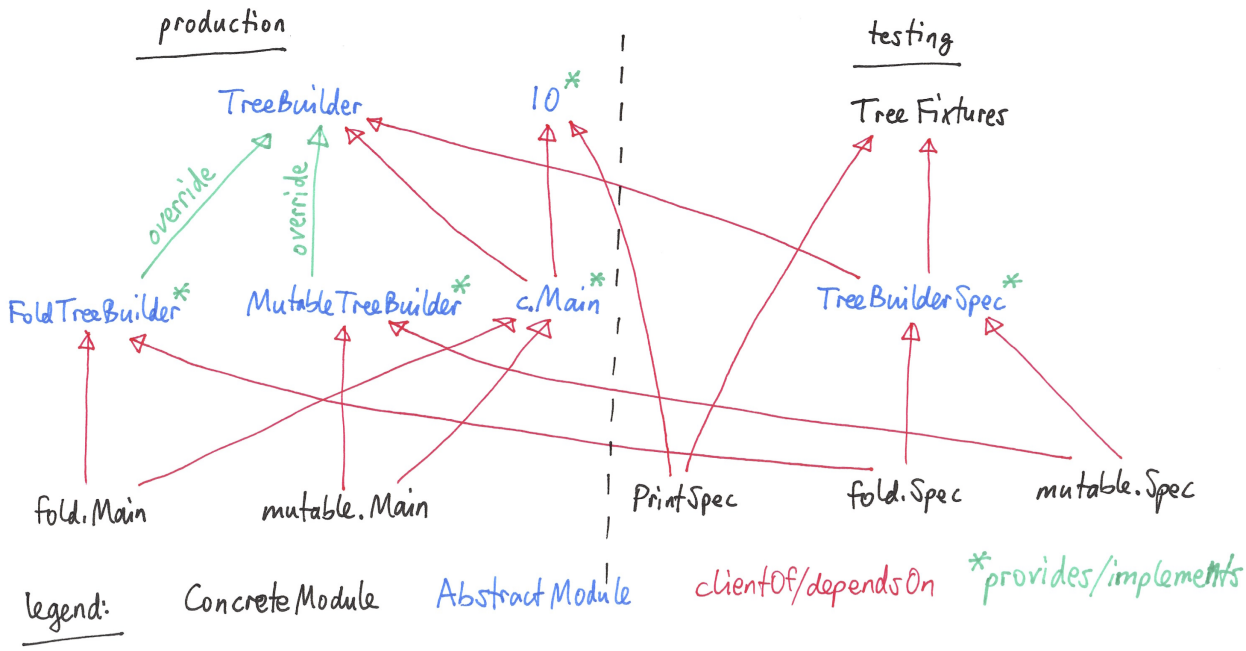
To avoid code duplication in the presence of multiple design dimensions, we can again leverage Scala traits as building blocks. Along some of the dimensions, there are three possible roles:

- *provider*, e.g., the specific implementations *MutableTreeBuilder*, *FoldTreeBuilder*, etc.
- *client*, e.g., the various main objects on the production side, and the *TreeBuilderSpec* on the testing side
- *contract*, the common abstraction between provider and client, e.g., *TreeBuilder*

Usually, when there is a common contract, a provider *overrides* some or all of the abstract behaviors declared in the contract. Some building blocks have more than one role. E.g., `common.Main` is a client of (depends on) `TreeBuilder` but provides the main application behavior that the concrete main objects need. Similarly,

`TreeBuilderSpec` also depends on `TreeBuilder` but provides the test code that the concrete test classes (`Spec`) need. This arrangement enables us to mix-and-match the desired `TreeBuilder` implementation with either `common.Main` for production or `TreeBuilderSpec` for testing.

The following figure shows the roles of and relationships among the various building blocks of the process tree example.



Dependency injection (DI) is a technique for supplying a dependency to a client from outside, thereby relieving the client from the responsibility of "finding" its dependency, i.e., performing *dependency lookup*. In response to the popularity of dependency injection, numerous DI frameworks, such as Spring and Guice, have arisen.

The Thin Cake idiom provides basic DI in Scala without the need for a DI framework. To recap, `common.Main` cannot run on its own but declares by extending `TreeBuilder` that it requires an implementation of the `buildTree` method. One of the `TreeBuilder` implementation traits, such as `FoldTreeBuilder` can satisfy this dependency. The actual "injection" takes place when we inject, say, `FoldTreeBuilder` into `common.Main` in the definition of the concrete main object `fold.Main`.

CHAPTER

FOUR

THE FUNCTIONAL PROGRAMMING PARADIGM

In this chapter, we study the functional programming paradigm, with examples and projects mostly in Scala.

4.1 Solving problems using built-in types and behaviors

As do other languages, Scala provides an extensive library of predefined types and (generic) type constructors along with a rich set of behaviors. Many of these, especially collection types and certain utility types, are *algebraic data types*, discussed below in more detail:

- `Seq` / `List`
  - http://scala-lang.org/api/current/index.html#scala.collection.immutable.List
  - https://github.com/lucproglangcourse/misc-explorations-scala/blob/master/lists.sc
  - note the difference between those and tuples
- `Map`
  - http://scala-lang.org/api/current/index.html#scala.collection.immutable.Map
- `Option` / `Either`
  - https://github.com/lucproglangcourse/misc-explorations-scala/blob/master/option.sc
  - https://github.com/lucproglangcourse/misc-explorations-scala/blob/master/either.sc
  - http://robsscala.blogspot.com/2012/06/fixing-scalaeither-unbiased-vs-biased.html
- `Try`
  - http://scala-lang.org/api/current/index.html#scala.util.Try
  - https://github.com/lucproglangcourse/misc-explorations-scala/blob/master/try.sc

By using Scala like a scripting language (such as Python or Ruby), one can solve many problems without even defining custom algebraic data types, except perhaps the occasional tuple. The main building blocks in scripting-style Scala are the collection and utility types we just mentioned, along with

- key methods `map`, `filter` / `withFilter`, `find`, `flatMap`, `sum`, `fold`, `groupBy`, `collect`
- `for` comprehensions

Todo

elaborate more on `for` comprehensions and `flatMap`

4.1.1 Examples

Loop over a finite collection or iterator using mutable state:

```
final Iterator<String> incoming = ...;
int sum = 0;
int count = 0;
for (final String s: incoming) {
  sum += s.length();
  count += 1;
}
final float result = (float) sum / count;
```

Immutable equivalent using `foldLeft`:

```
val (sum, count) = incoming.foldLeft {
  (0, 0)
} { case ((sum, count), next) =>
  (sum + next.length, count + 1)
}
val result = sum.toFloat / count
```

Unbounded loop until a condition is met:

```
while ((/* ... */ ; line = reader.readLine()) != null ; line) {
  processExpr(line)
}
```

Immutable equivalent using `continually`:

```
Iterator continually {
  // ...
  reader.readLine()
} takeWhile { line =>
  line != null
} foreach { line =>
  processExpr(line)
}
```

Note that we are using `foreach` when the body of the iteration produces a *side effect* such as output. If we wanted to compute a *result value*, we could use `foldLeft` instead of `foreach`.

Note also that all of these are methods but look like control structures because of Scala's syntax, which allows you to omit the dot in certain cases of method selection and to use curly braces instead of round parentheses to delimit your argument list.

The more familiar one becomes with the various predefined building blocks, the more quickly and productively one can put together at least an initial solution to a problem. Earlier versions of the process tree example illustrates this style, while later versions reflect greater emphasis on code quality, especially testability and avoidance of code duplication.

Todo

add more recent idioms including successive `Try` and `for` with blocks for embedding stateful steps such as logging

---

4.1.2 Challenges

Can we write (efficiently or not)

- `length`, `sum`, `reverse`, `filter`, `find`, `map` as a fold, i.e., `foldLeft` or `foldRight`?
- `foldLeft` or `foldRight` as `map`?!?
- `reverse` or `filter` as a `map`?

Some hints:

- Look carefully at the respective domains and codomains (argument and result types). Can they fit?
- Which is more general, `map` or `fold`?

4.2 Defining algebraic data types

Most structures fall into one of these categories:

- nonrecursive/scalars: boolean, finite enumerations (including numeric types), try
- sublinear structures: (infinite set of) natural numbers, option
- linear structures: lists, maps
- nonlinear structures: trees, graphs, many custom domain models

The fundamental building blocks of these *algebraic data types* are related to those discussed in *Defining domain models in imperative and object-oriented languages*:

- (disjoint) sum: variation
- product (tuple, record) of a given arity: aggregation
- recursion (at the type level)
- type parameters (genericity)

Using these building blocks, we can express the *Shape* domain model from the examples above as an algebraic data type:

```
Shape = Circle(Int)
      | Rectangle(Int, Int)
      | Group(Seq(Shape))
      | Location(Int, Int, Shape)
```

We can separately define behaviors on Shapes as functions. Here is an example that illustrates this approach:

- https://github.com/lucproglangcourse/shapes-oo-scala
- https://github.com/lucproglangcourse/expressions-scala

We identify the following structural and behavioral concerns:

- structure
- content
- traversal
- processing

So far, structure and content are combined within the definition of an algebraic data type, while traversal and processing are combined within the definition of a behavior on that algebraic data type.

4.2.1 Separation of structural concerns

We can, however, achieve a separation between structure and content with the help of *parametric polymorphism*, that is, making the algebraic data type *generic* in terms of the content. The predefined collections are an example of this separation, as well as the generic org chart example.

4.3 Behaviors on algebraic data types

The following are additional examples of behaviors on algebraic data types. As expected, for recursive types, the behaviors are typically recursive as well.

- simple org charts
- generic org charts
- simple natural numbers
- expressions-scala

In these examples, the traversal and processing concerns identified above remain combined.

4.3.1 Separation of behavioral concerns

A question that comes to mind is whether they can be separated, similarly to the predefined higher-order methods on collections, such as `foldLeft`, `foldRight`, `map`, etc. These methods go a step further than the Visitor pattern or our equivalent recursive behaviors: They handle the *traversal* concern for us and separate it from the *processing* concern, which we handle by providing a suitable argument function.

- functions on lists (reverse)
- functions on streams (potentially infinite lists with memoization)

This question has a two-part answer: Yes, we can define custom implementations of such higher-order behaviors for our own algebraic data types. In addition, and this is where it gets really interesting, we can have a single, universal implementation that works for all algebraic data types where the children of any node are either fixed in number or stored in a collection that has a `map` method.

Another, seemingly esoteric, question is whether we can pull out recursion itself as a functional pattern. Yes, we can. In this factorial example, the `Y`-combinator handles the *recursion* concern *for behaviors* and separates it from the concern of what should happen in each step of the recursion.

We will soon study the equivalent idea at the type level.

4.4 A closer look at predefined behaviors on lists

In this section, we take a look "under the hood" of some key predefined behaviors on lists.

In terms of performance, we must keep in mind that lists are head/tail-optimized. In other words, these are basically singly-linked lists, so any behaviors where we access the first node of the list are constant-time, while behaviors involving nodes further down in the list are linear-time. In practice, acceptable performance usually means linear time for behavior where we process the entire list.

In addition, we need to be aware of *space complexity*. Clearly, we are already using space for the arguments we are about to pass to the behavior and are willing to dedicate space to the result we are getting back, so the focus is on *additional* temporary space on the stack, which we like to keep constant if possible. (This discussion is closely related

to *The importance of constant-space complexity*, where the assumption is that the arguments and the result are stored *externally*.)

Tail recursion, where the very last step in a method or function body is the recursive invocation of the method itself, is an effective technique for achieving constant-space complexity as long the behavior can be expressed in a tail-recursive way. In some cases, we can rewrite an implementation in a tail-recursive way by introducing an *accumulator* argument, where we essentially build up the result in the accumulator and then return that result once we reach the base case of the recursion. A tail-recursive implementation can easily be transformed to a `while` loop by introducing a mutable variable to represent the progress into the list structure. This reverse example illustrates these concepts and techniques in more detail.

Here are some observations:

- `foldLeft` is usually what we want: *linear-time* and *constant-space* (naturally tail-recursive).
- `foldRight` is *linear-time* and *linear-space* (*not* tail-recursive) but goes with the natural head-tail structure of the list.
- *xs.foldRight(z)(f) == xs.reverse.foldLeft(z)(g)* where *g* is *f* with the arguments switched.

To look at the actual Scala library implementations of these functions, first find desired method in the API documentation, expand, look for *definition classes*, follow the link to the leftmost definition class, then the link to that class's Scala source, and finally look for the actual method. For performance reasons, these professional implementations tend to appear more complex than we might expect. Here are some examples:

- foreach
- foldLeft
- reverse
- foldRight
- map
- length

For more details on space complexity and tail recursion, please take a look at these references:

- space complexity, tail recursion, and accumulators
- tail recursion, trampolines, and continuations (advanced)

4.5 Separation of concerns at the type level

The overall approach is to separate recursion from structure by formalizing algebraic data types as initial F-algebras.

4.5.1 Key concepts

We first need to define some key concepts:

- (Endo)functor: a type constructor (generic collection) with a `map` method that satisfies *identity* and *composition* laws:

  ```
  c.map(identity) == c
  c.map(g compose f) == c.map(f).map(g)
  ```

  Some familiar examples of endofunctors are

  - `Option`

  - `List`
  - generic trees such as org chart
- The `Fix`-combinator handles the *recursion* concern *for structures* and separates it from the nature of the structure itself.
- Generalized `fold` = *catamorphism* (`cata`) for *breaking down* a data structure to a result value.
- F-algebra: This is the argument to `fold`, which has a functor `F` and a carrier object, i.e., the result type of the fold.
- `unfold` = *anamorphism* for *building up* a data structure from some other value.
- *F-coalgebra*: This is the argument to `unfold` (generator), which also has a functor `F` and a carrier object, i.e., type of seed and generated values wrapped in the functor.
- *Initial F-algebra*: This is the least fixpoint of our functor `F` and equivalent to our original recursive type. We obtain this by applying the `Fix`-combinator to `F`.
- We get our original recursive behaviors back by combining `cata` and our specific F-algebraic version of the behavior.

Todo

Practical applications

4.5.2 Examples

It is perhaps best to look at some conventional and F-algebra-based examples side-by-side:

- expressions-scala versus expressions-algebraic-scala
- project 2a versus project 2b

Some other examples are available here.

4.5.3 What `Fix` does

`Fix[F]` basically ties the "recursive knot" by applying the functor `F` to itself. This forms the *fixpoint* of the functor, allowing all structures built from the functor to have the same type, as opposed to nested types corresponding to the nesting of the structure.

For instance, we can represent the familiar aggregation of an item and an (optional) next node using the functor `F[A] = (Int, Option[A])`. This enables us to define linked lists:

```
(1, Some((2, Some((3, None)))))
```

The problem is that the types of these lists are nested:

```
scala> (1, Some((2, Some((3, None)))))
res0: (Int, Some[(Int, Some[(Int, None.type)])]) = (1,Some((2,Some((3,None)))))
```

so that lists of different lengths have different types.

By using a suitable `Fix` over our functor, they all end up having the *same* type, namely `Fix`:

```
case class Fix(unFix: (Int, Option[Fix]))

scala> Fix((1, Some(Fix((2, Some(Fix((3, None))))))))
res1: Fix = Fix((1,Some(Fix((2,Some(Fix((3,None))))))))
```

That's why we usually define such types recursively to begin with.

4.5.4 Generalized fold (catamorphism)

The next question is what the implementation of the universal fold method for `Fix` looks like, also known as the *catamorphism*. Continuing with our `Fix` over `(Int, Option[A])` example, we perform recursion over this functor by using `map`, which preserves the first component and invokes a suitable `map` on the second component of the pair:

```
case class Fix(unFix: (Int, Option[Fix])) {
  def cata[B](f: ((Int, Option[B])) => B): B = f((this.unFix._1, this.unFix._2.map(_.
↪cata(f))))
}
```

Now we can define *algebras* on our functor, such as:

```
def sum(arg: (Int, Option[Int])): Int = arg match {
  case (i, None) => i
  case (i, Some(s)) => i + s
}

res1.cata(sum) // 6
```

These are very similar to visitors without the responsibility to traverse the structure. That is why they are not recursive. Instead, the catamorphism takes care of the recursion.

For an arbitrary functor `F`, the code looks like this:

```
case class Fix(unFix: F[Fix]) {
  def cata[B](f: F[B] => B): B = f(this.unFix.map(_.cata(f)))
}
```

For an arbitrary *carrier type* `B`, the argument `f` of type `F[B] => B` is an `F`-algebra. `Fix[F]` is the *initial* `F`-algebra, and the catamorphism `cata` produces the unique structure-preserving mapping (homomorphism) between `Fix[F]` and `f`.

4.5.5 Key insights

By taking an F-algebraic perspective on recursive algebraic data types, we are able to recognize previously non-obvious structural commonalities among them.

- non-generic: `Nat`, `Expr`, `Shape`, etc.
- generic: `List`, `Tree`, `OrgChart`, etc.

It also helps to study these questions:

- How are, say, `Option`, `List`, and `Tree` related?
- How does
  - `Option` relate to `List`
  - `List` relate to `Tree`

  - `Tree` relate to ?!?
  - ...

- How do we represent an *empty* structure?
- Why aren't there multiple branches in the definition of `cata` above? When does the recursion terminate?
- Is `cata` tail-recursive? Can or should it be?

On the behavioral side, we recognize the great potential for code reuse resulting from common abstractions:

- Scalaz library
- Matryoshka library
- The various Typelevel.scala projects

For more details on F-algebras and datatype-generic programming, please take a look at these references:

- Advanced Functional Programming with Scala
- Understanding F-Algebras
- Gibbons: origami programming (advanced)
- Oliveira & Cook: F-algebras in Java (advanced)

If you want to dig a bit deeper, check out a generalization of `map` called traverse. Some of our examples include implementations of `traverse`.

4.6 Other useful abstractions

In this subsection, we will discuss a few more useful yet relatively simple abstractions.

4.6.1 Monoid

A Monoid is a type with an associative binary operation and an identity element. (This is equivalent to a semigroup with an identity element.) Examples include:

- integers with addition and zero
- integers with multiplication one
- lists with append and the empty list
- strings with concatenation and the empty string

The *monoid laws* arise from the monoid's definition: the operation must be associative, and the identity element must be a left and right identity.

Examples of monoids using the Scalaz library are available here

4.6.2 Monad

A Monad is a type constructor (generic collection) with two operations, `point` (also called `return` or `unit`) and `flatMap` (also called `bind`). Monads are an effective way to represent the *context* of a computation in which the computation is "wrapped". The monad abstraction thereby enables one to separate the concerns of the computation itself and its context. Examples include:

- `Option` and `Try`: potential failure in a computation

- `List`: nondeterminism in a computation, meaning that the computation might have multiple results
- `Id`: the identity monad, a wrapper that doesn't actually do anything
- `Future`: the computation takes place asynchronously (in the background)

Examples of monads using the Scalaz library are available here.

4.6.3 Observations

- The Scala library includes various structures that are effectively monads, especially those just mentioned. What Scala does not define is a monad abstraction itself.
- This is where libraries like Scalaz or Cats come in: They define these abstractions in such a way that we can retrofit existing types or our own types to become instances of the desired abstractions, using the *Typeclass pattern*, a technique for representing Haskell-style typeclasses.
- Examples of the Typeclass pattern are the `Functor` and `Traverse` instances in our expressions and shapes examples.
- A good reference for learning Scalaz, a library that defines these various abstractions, is available here.

4.7 References

Todo

put chapter-level references here

CHAPTER

FIVE

PROGRAM REPRESENTATION AND INTERPRETATION

In this chapter, we discuss how we can represent and ultimately execute or interpret programs in a particular language. To this end, we need to talk about the toolchain required to do so, as well as techniques for defining the meaning (semantics) of programs in our language.

5.1 Programming language toolchains

We will start with an overview of the phases of programming language compilation/interpretation toolchain:

- source code (string stored in file)
- *lexical analysis*
- sequence of tokens
- *syntax analysis (parsing)*
- abstract syntax tree <- interface between front end and back end
- *static checks*
- *optimization*
- transformed abstract syntax tree
- *code generation*
- machine code or byte code
- *execution* on physical hardware, virtual hardware, or virtual machine
- alternative: *direct interpretation of abstract syntax trees*

Reference: Mogensen

5.1.1 Cross-compilation

Cross-compilation is a useful technique when the target environment does not provide sufficient computational resources to compile software in the target environment itself. Common examples include mobile phones and other embedded devices. For example, one usually develops Android apps on a development machine, tests them on an Android virtual device (emulator), and deploys them to an actual phone or tablet.

The following example shows cross-compilation of a small C program on a development machine and execution in the target environment.

1. First, we compile the source on our development machine, in this case, a hosted Linux VM using the OpenWrt embedded Linux toolchain:

```
[laufer@boole:~/Work/hello] 130 $ cat hello.c
main() {
  puts("hello ETL!");
}
[laufer@boole:~/Work/hello] $ ~/Work/OpenWrt-SDK-15.05.1-ramips-rt305x_gcc-4.8-
↪linaro_uClibc-0.9.33.2.Linux-x86_64/staging_dir/toolchain-mipsel_24kec+dsp_gcc-
↪4.8-linaro_uClibc-0.9.33.2/bin/mipsel-openwrt-linux-uclibc-gcc hello.c
```

2. Now we deploy the resulting executable to the target environment using, say, `scp`.
3. Finally, we run this executable natively on a MIPS-based embedded Linux device sold as a "portable travel router":

```
root@ExPodNano:~# ldd ./a.out
libgcc_s.so.1 => /lib/libgcc_s.so.1 (0x770d5000)
libc.so.0 => /lib/libc.so.0 (0x77069000)
ld-uClibc.so.0 => /lib/ld-uClibc.so.0 (0x770f9000)
root@ExPodNano:~# ./a.out
hello ETL!
```

5.2 Compiler front-end concepts and techniques

In this section, we will discuss the key front-end stages, lexical analysis and syntax analysis, and related concepts and techniques.

5.2.1 Lexical analysis

- regular expressions, NFA, DFA (Mogensen ch. 2)
- regular expressions
- regular expressions in Java

5.2.2 Syntax analysis

- context-free grammars, EBNF, parsing (Mogensen ch. 3)
- parser generator
- scala-bison

5.2.3 Alternative front-end approaches

- parser combinators
- Scala parser combinators
  - Odersky et al. ch. 33
  - API
  - simple expression combinator parser example
- parsing expression grammars

- From EBNF to PEG
- Parboiled 2 PEG library
  - parboiled2 documentation
  - parboiled2 JSON parser example

5.3 Programming language semantics

Semantics, a formalization of the *meaning* of programs in a particular language, includes *static* and *dynamic* semantics.

- static semantics (compile-time)
  - flow analysis
  - typing
- dynamic semantics (run-time)
  - denotational
  - operational, e.g. our interpreters
  - axiomatic

The Interpreter pattern is related to the discussion of dynamic semantics.

- http://c2.com/cgi/wiki?InterpreterPattern
- http://en.wikipedia.org/wiki/Interpreter_pattern

5.4 Case study: a simple imperative language

In this section, we discuss the design of an interpreter and programming environment for a simple imperative language.

- Java implementation
- Scala implementation

The accompanying lecture is available as a set of screencasts:

- foundations
- implementation

5.4.1 Objectives

We are leading toward building an interpreter for a language resembling JavaScript with object-like features. We are doing this with two aims in mind:

- To understand an object-oriented programming language from the "inside".
- To understand better the object-oriented and functional design patterns that we have learned by using them in the context of a small but somewhat sophisticated program.

5.4.2 Syntax

Our language has the following features:

Integer expressions given by the BNF grammar:

```
e ::= variable
  |   const
  |   e1 + e2
  |   e1 - e2
```

Statements given by the BNF grammar:

```
S ::= x = e
  |   S1 ; S2
  |   while (e) S
```

In the context of the lectures so far, we have made the following changes. We have added variables to expressions, thus we can handle cases like *x + 3*, whereas earlier we could only write expressions such as *4 + 3*. We have also introduced the assignment statement as a way to change the contents of a variable. In addition, we allow statements to be put in sequence. We also permit simple while expressions, where the guard is an expression and the loop body is executed while the gurad expression evaluates to a non-zero integer value.

5.4.3 Structured operational semantics (SOS)

We first formalize the intuitive execution semantics of the toy language. The point of doing this is to present the basic ideas in the interpreter without getting tied up in the programming details of the interpreter. In any case, these details are presented later in this lecture. Since our toy language has variables, we need to keep track of the state of variables. We view variables as objects with two capabilities:

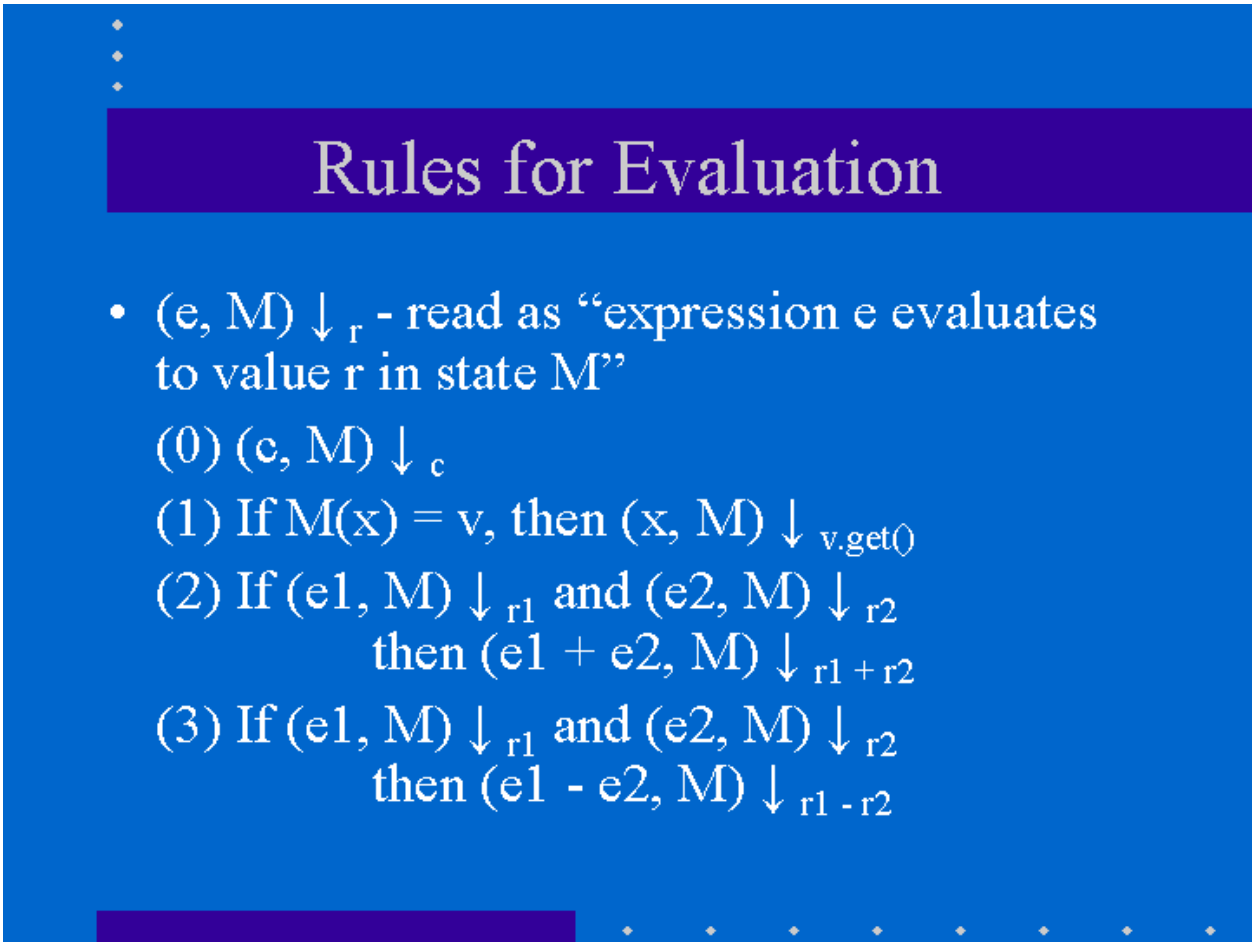
- *v.get()* returns the current value of the variable
- *v.set(int i)* changes the current value of the variable to that of *i*

We think of the state of the program (memory store), which we write *M*, as a map that associates identifiers with variable objects.

The rules for evaluating expressions are quite simple.

- Evaluating constant *c*. Every constant evaluates to itself.
- Evaluating a variable whose name is *x*: Retrieve the variable object (say *v*) associated with *x* from the memory store *M*, by using *M(x)*. The required result is computed by invoking *v.get()*.
- Evaluating *e1 + e2*: Evaluate *e1* first, say to yield value *r1*. Evaluate *e2* next, say to yield value *r2*. The required result is the numeric value *r1 + r2*.
- Evaluating *e1 - e2*: Evaluate *e1* first, say to yield value *r1*. Evaluate *e2* next, say to yield value *r2*. The required result is the numeric value *r1 - r2*.

The evaluation rules are written out precisely in the following picture.



The rules for executing statements is as follows. In contrast to expression evaluation, statement execution does not yield a result. The primary consequence of executing a statement is the side effect on the store, ie. changes in the values of variables.

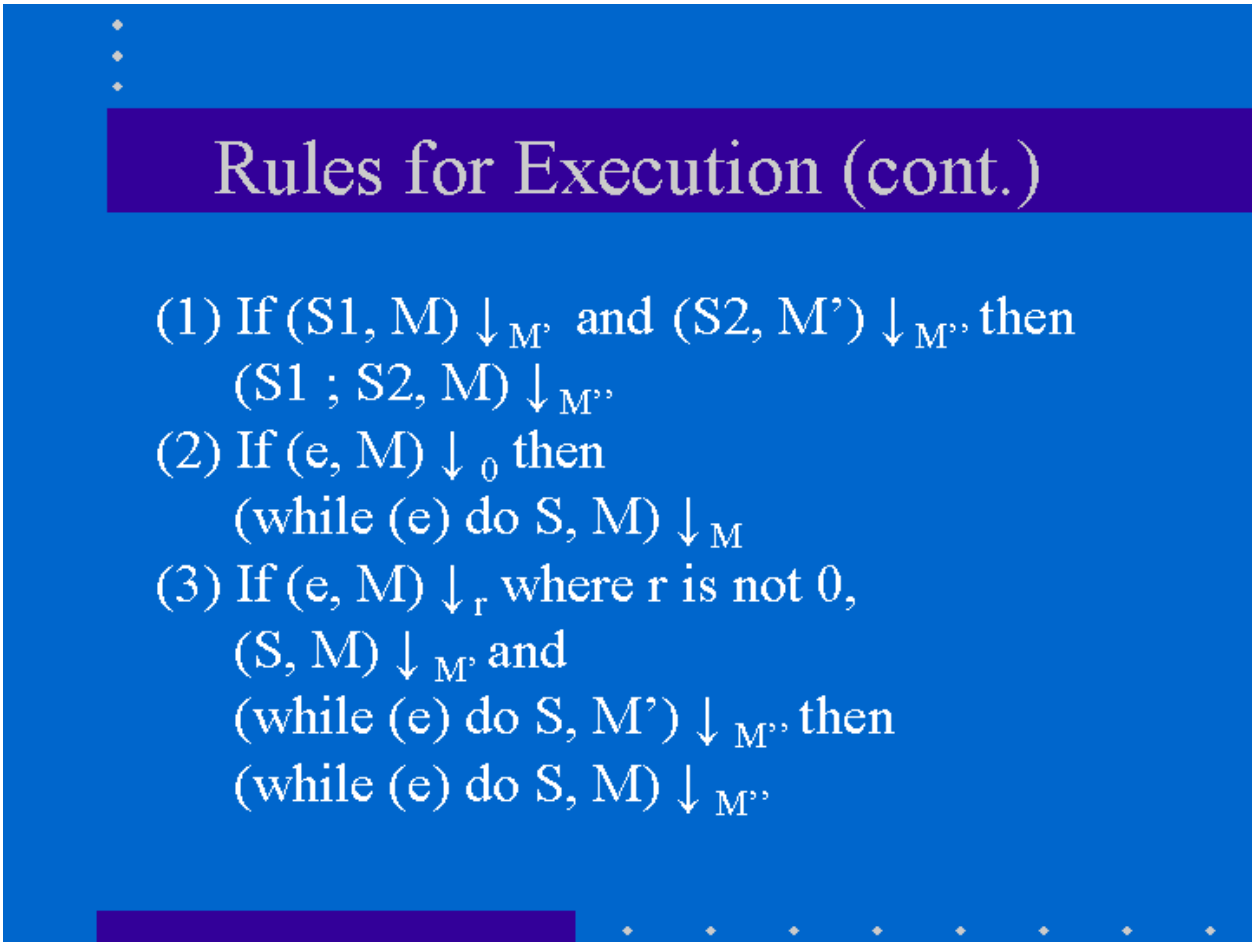
- Executing an assignment statement: Consider the assignment statement $x = e$. Here e is an expression. The steps are as follows: First, evaluate the expression e to yield a result, say r. Next, retrieve the variable object (say v) associated with x from the memory store M, by using $M(x)$. Perform *v.set(r)* to change the value of the variable object.
- Executing a sequence of statements *S1; S2*: Execute *S1* first. When that terminates, execute *S2*.
- Executing *while (e) do S*: Evaluate the condition e to yield a result r. If r is zero, the execution terminates. Otherwise, execute S and repeat the process.

The execution rules are given in the following pictures:

Rules for Execution

- $(S, M) \downarrow_{M'}$ - read as " executing statement S in state M results in state M' "

  (0) If $(e, M) \downarrow_{r}$ then $(x = e, M) \downarrow_{M'}$
  where M' is the map given by:
  $M'(x) = v$ such that $v.get() = r$
  $M'(y) = M(y)$, if y is not x



Note that the connection between the various statements is that they share a single store, ie. in the sequence of statements *x = 2; y = x + 1*, the second reference to *x* reflects the effect of the first assignment because of the (shared) store between the two assignment statements.

5.4.4 The interpreter program

We now go ahead and write the interpreter program. The entire code is available as part of the misc-scala example. A similar example but based on F-algebras and with better runtime error handling is also available: simpleimperative-algebraic-scala.

5.5 Case study: a simple imperative language with records

In this section, we discuss the design of an interpreter and programming environment for a simple imperative language with records (like classes with public fields and without methods).

5.5.1 Objectives

We are leading toward building an interpreter for a language resembling JavaScript with object-like features. We are doing this with two aims in mind:

- To understand an object-oriented programming language from the "inside".

- To understand better the object-oriented and functional design patterns that we have learned by using them in the context of a small but somewhat sophisticated program.

We now consider the changes that arise from the addition of records. Thus, we permit:

- declaration of record types
- creation of new records of a given record type
- selection of record fields
- use of records on the left and right hand side of expressions

5.5.2 Syntax

Our language has the following features:

The syntactic feautures of our language are captured by the following grammar. For motivation, the sort of program that we are interested is exemplified by:

```
StudentCourseRecord = record
    int firstExamScore;
    int secondExamScore;
    int totalScore;
end;

StudentSemRecord = record
    StudentCourseRecord course1;
    StudentCourseRecord course2;
end;

StudentSemRecord r = new StudentSemRecord();
r.course1 = new StudentCourseRecord();
r.course1.firstExamScore = 25;
r.course1.secondExamScore = 35;
r.course1.totalScore = r.course1.firstExamScore + r.course1.secondExamScore;

r.course2 = r.course1;
```

In the C language, such things are known as structs. In familiar object-oriented terminology, we can think about them in this way:

- record types are classes whose only members are public member variables
- records are objects
- fields are public member variables

The record type definitions in the previous example would look as follows in Java, and the rest of program would look the same:

```
class StudentCourseRecord {
    public int firstExamScore;
    public int secondExamScore;
    public int totalScore;
}

class StudentSemRecord {
    public StudentCourseRecord course1;
    public StudentCourseRecord course2;
}
```

Formally, we proceed via the following BNF grammars. To simplify life for us, we will ignore type information. In this BNF grammar, we are a little bit more careful to separate L(eft) values and R(ight) values. L-values are those that can appear on the left hand side of an assignment statement, and R-values are those that appear on the right hand side of an assignment.

Record definitions are given by the BNF grammar:

```
Defn ::= record
             FieldList
         end


FieldList ::= fieldName, FieldList
           | fieldName
```

L-values (fields selected from records, as well as variables) are given by the BNF grammar:

```
Lval ::= e.fieldName
     | variable
```

Expressions (R-values) are given by the BNF grammar:

```
e   ::=     new C
    |       Lval
    |       const
    |       e1 + e2
    |       e1 - e2
```

Statements are given by the BNF grammar:

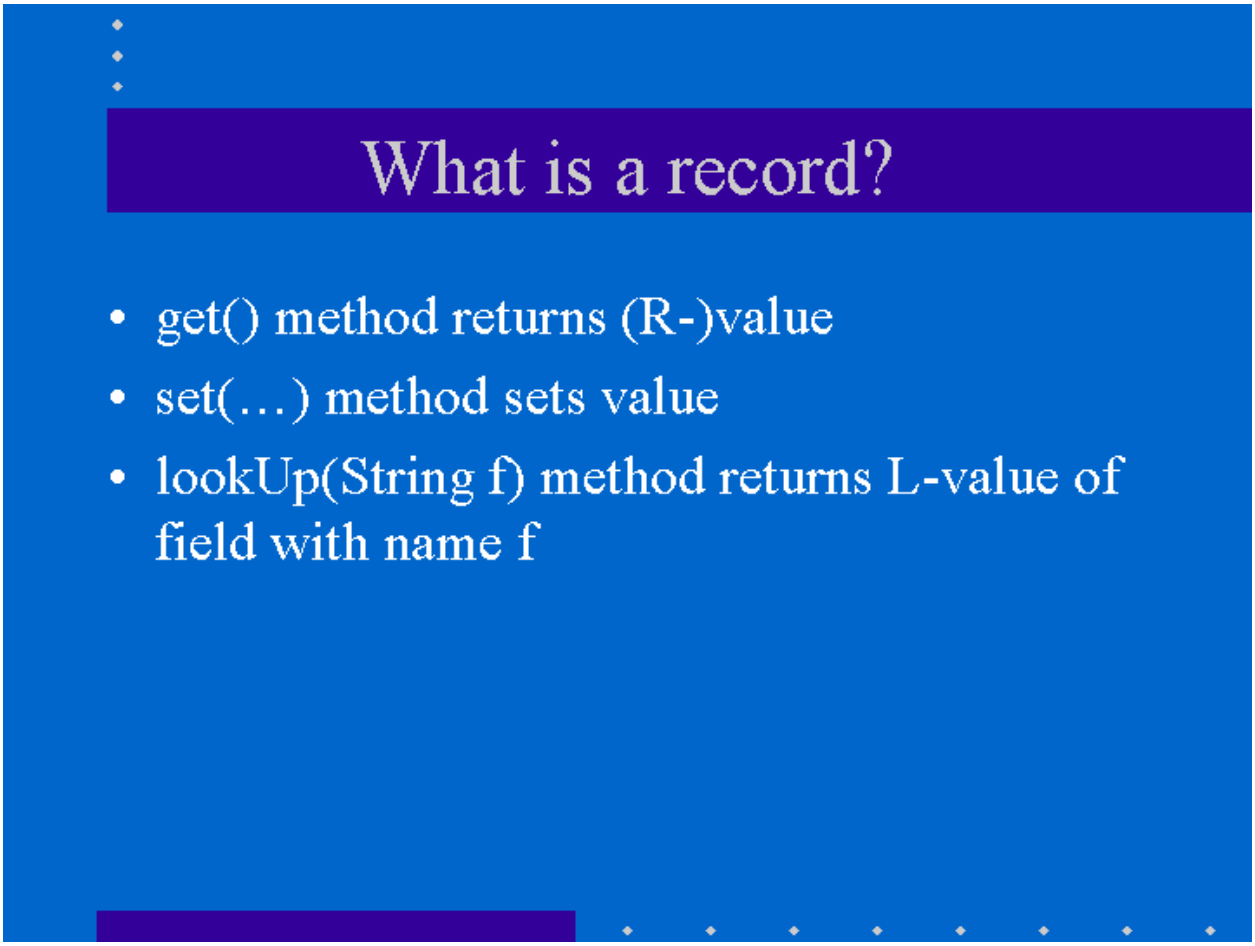
```
S   ::=     Lval = e
    |       S1; S2
    |       while (e) do S
```

We first formalize the intutive execution semantics of the toy language. As before, the point of doing this is to present the basic ideas in the interpreter without getting tied up in the programming details of the interpreter. In any case, these details are presented later in this lecture. In particular, in this initial first cut, we will begin by ignoring declarations. Also, in this new presentation

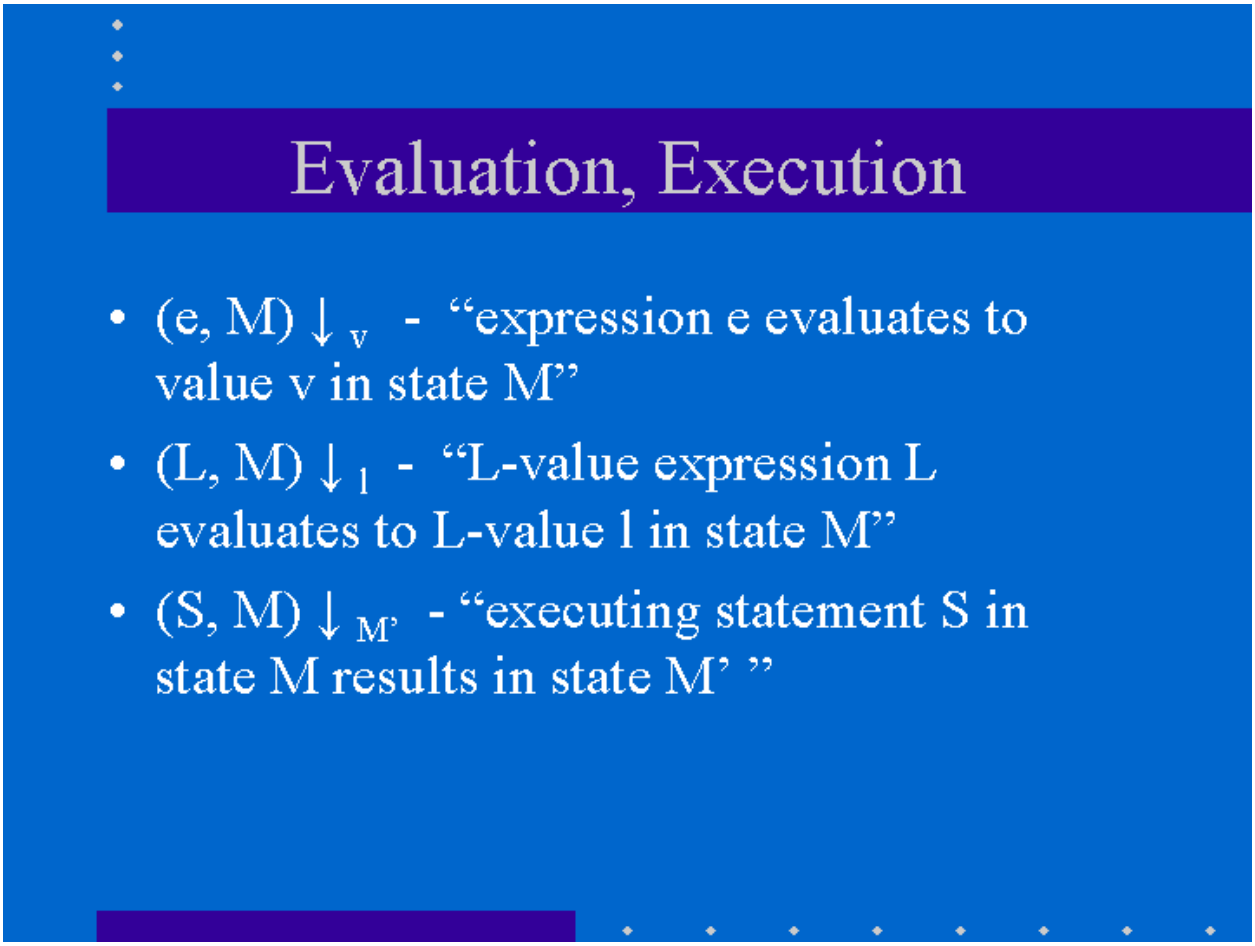
Recall that we viewed variables as objects with two capabilities:

- get() returns the current value of the variable
- set(int x) changes the current value of the variable to that of x

Records are thought of in a similar light.



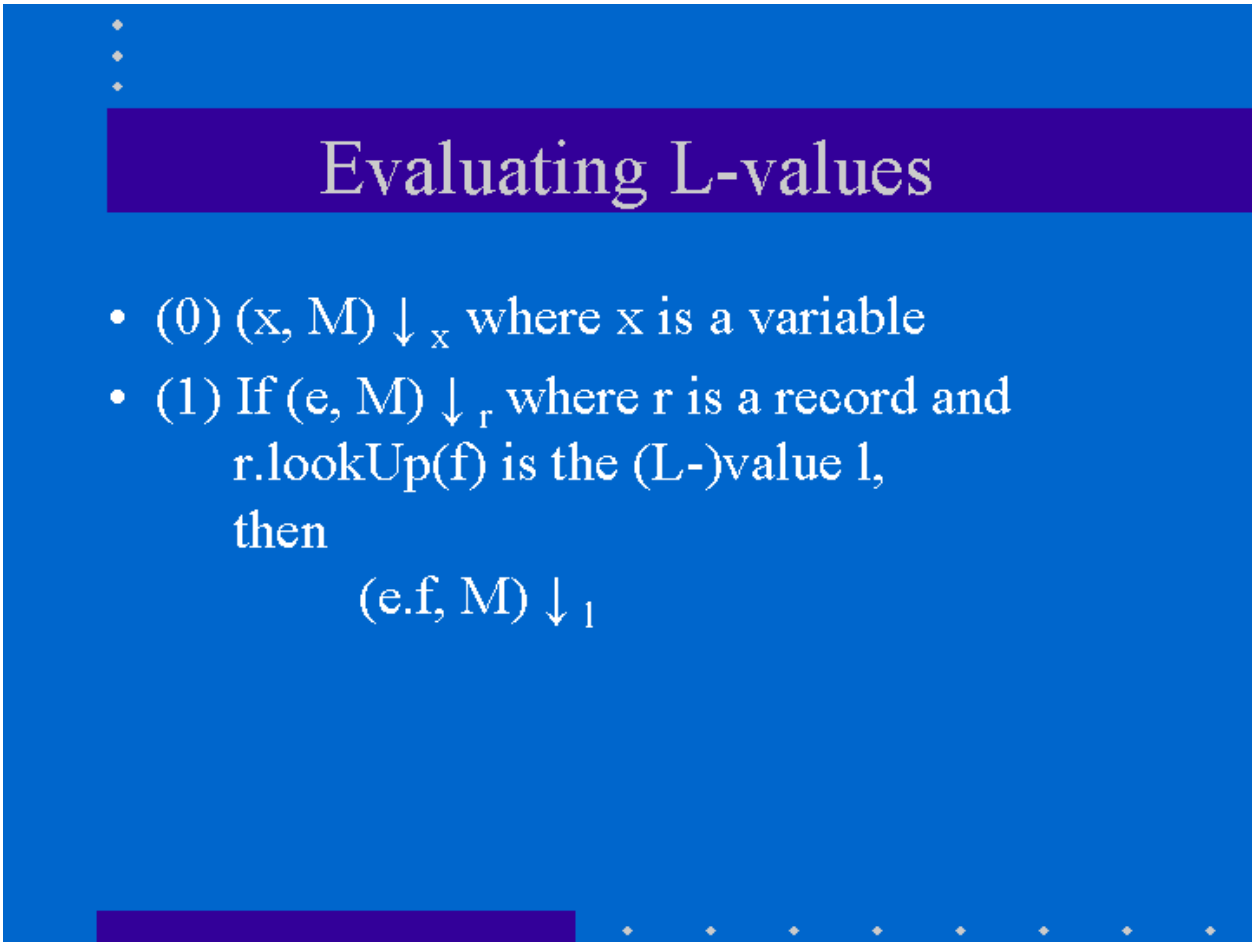
As before, we think of the state of the program, which we write S, as a map that associates identifiers with variable objects. Furthermore, as before, we distinguish evaluation and execution. In evaluation, there are two subcases, evaluating to an L-value and evaluating to an R-value.



There are two ways of having L-values. One is via variables and the second is via field selection.

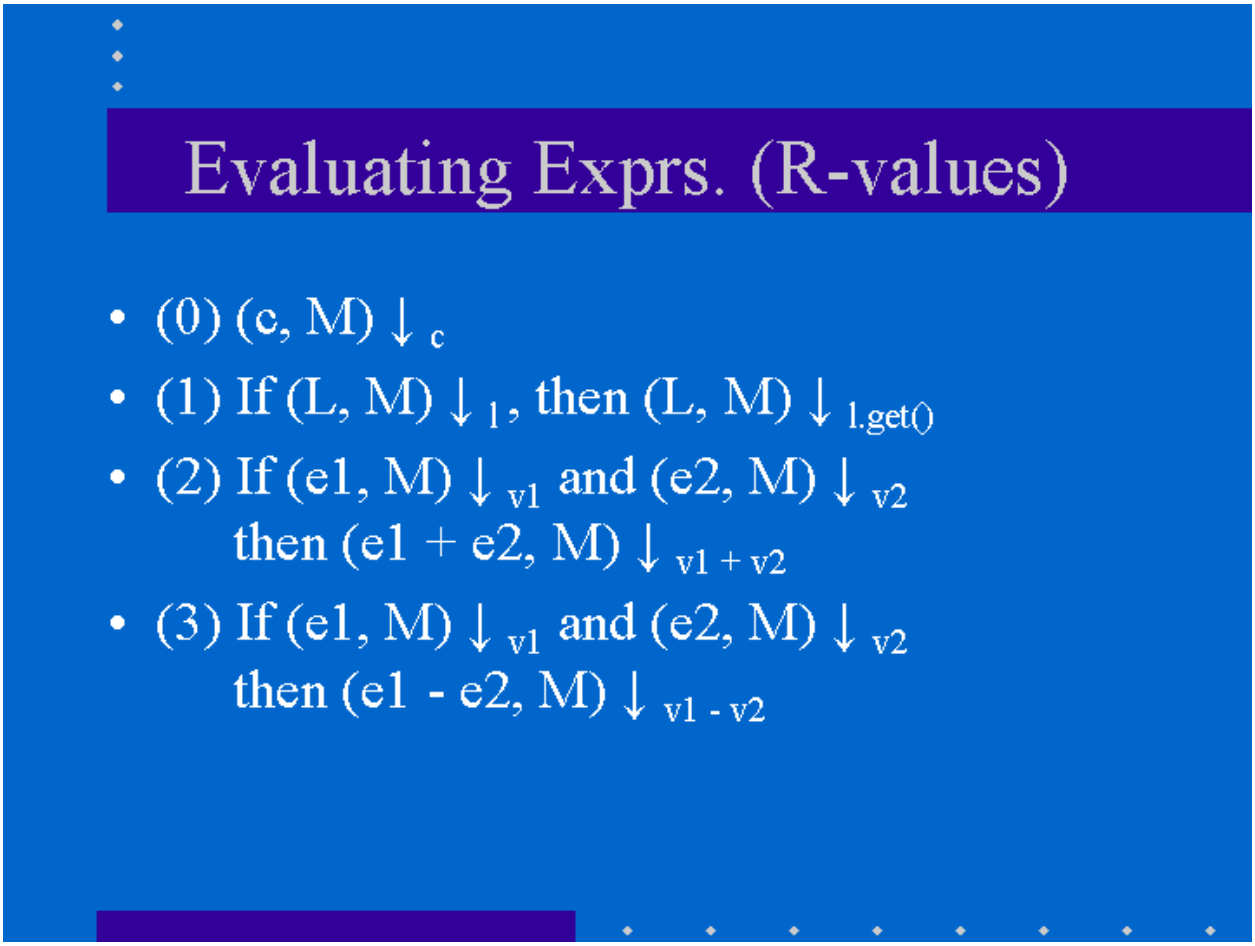
- The L-value associated with a variable name is the associated variable object.
- The L-value associated with a selection e.f is obtained by first evaluating the expression e to an R-value, say r. Next, lookup on the record r with field name f is used to get the desired L-value.

These evaluation rules are written out precisely in the following picture.



Our earlier rules for evaluating R-values are presented again below..

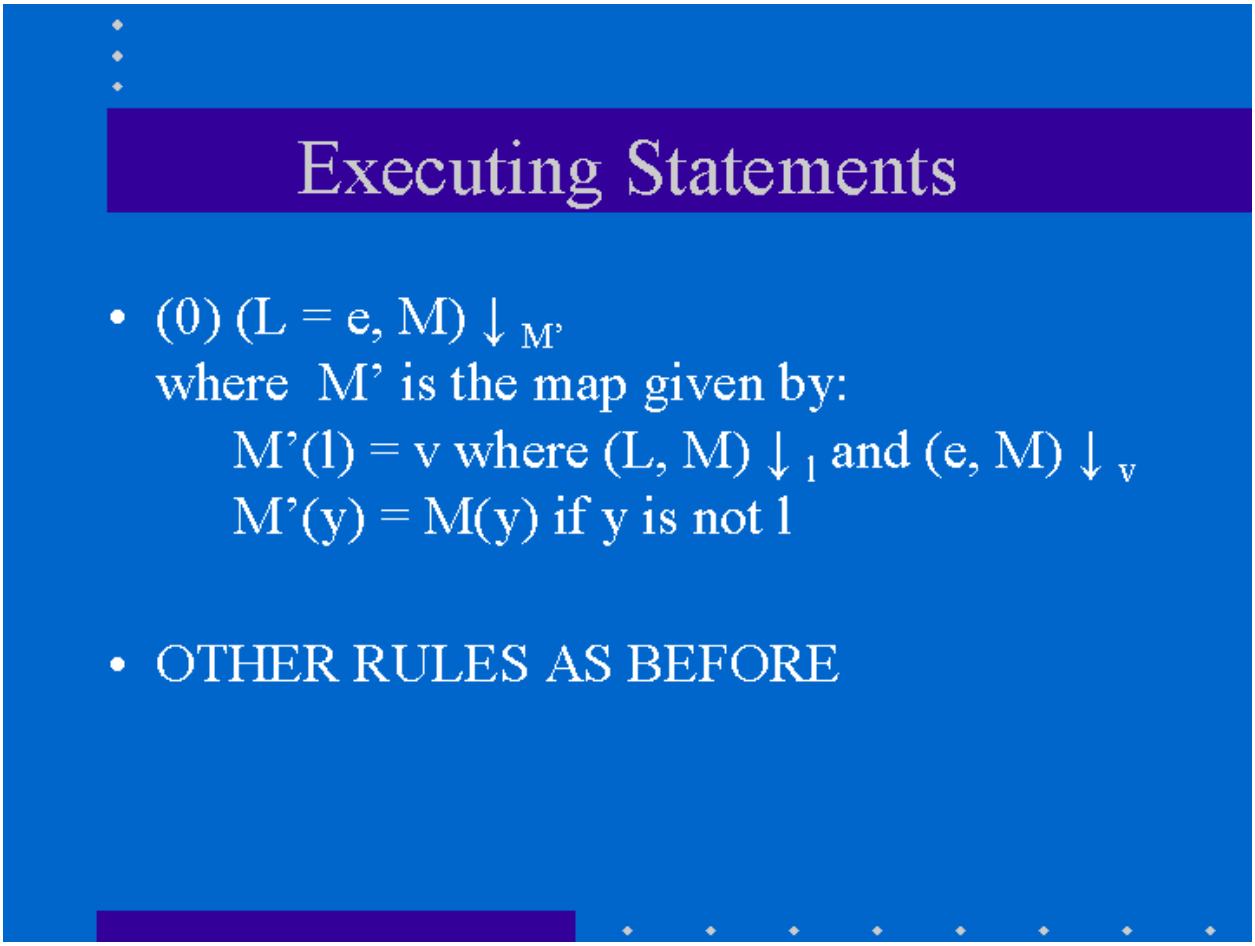
- Evaluating an L-value. In our setup, every L-value (say l) is a variable object that is obtained from the store M. Execute l.get() to compute the return value. This rule subsumes the earlier case for variables.
- Evaluating e1 + e2: Evaluate e1 first, say to yield value v1. Evaluate e2 next, say to yield value v2. The required result is v1 + v2.
- Evaluating e1 - e2: Evaluate e1 first, say to yield value v1. Evaluate e2 next, say to yield value v2. The required result is v1 - v2.
- Evaluating constant c. Every constant evaluates to itself.



The rules for executing statements are as follows. They are similar to the ones seen before. The primary consequence of executing a statement still is the side effect on the store, ie. changes in the values of variables.

- Executing an assignment statement L = e. Here L is an L-valued expression and e is an R-valued expression. The steps are as follows. First, evaluate the expression L to yield an L-value, say l. Next, evaluate the expression e to yield an R-value, say v. Next, use l.set(v) to change the value of the variable object.
- Executing a sequence of statements "S1; S2" and a "while" loop are as before.

The execution rules are given in the following pictures:



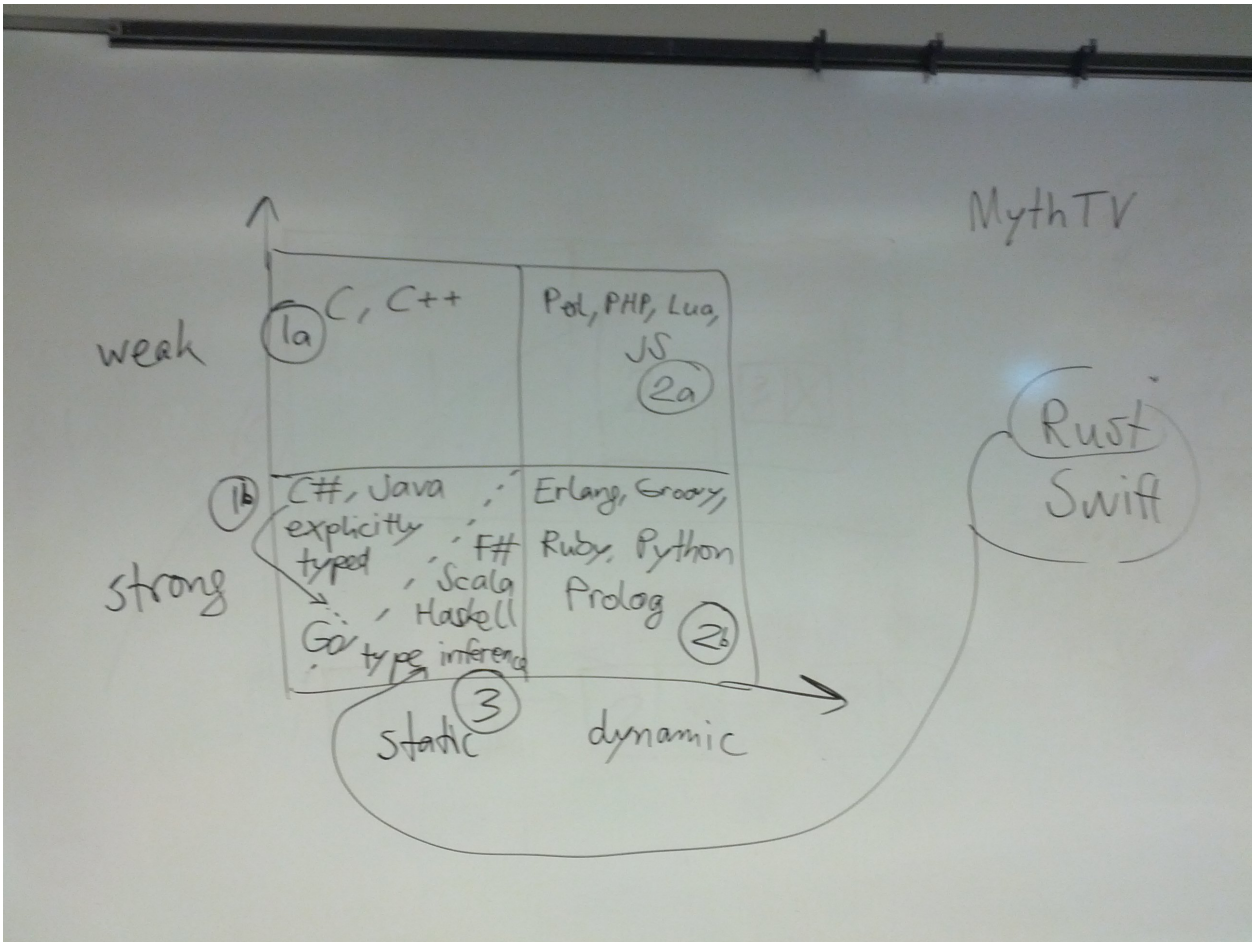
5.5.3 Implementation

The entire code for the implementation of the simple imperative language with records is available here.

5.6 Type systems

Type systems are an important aspect of programming languages. We identify the following dimensions:

- static versus dynamic
- strong versus weak
- implicit versus explicit



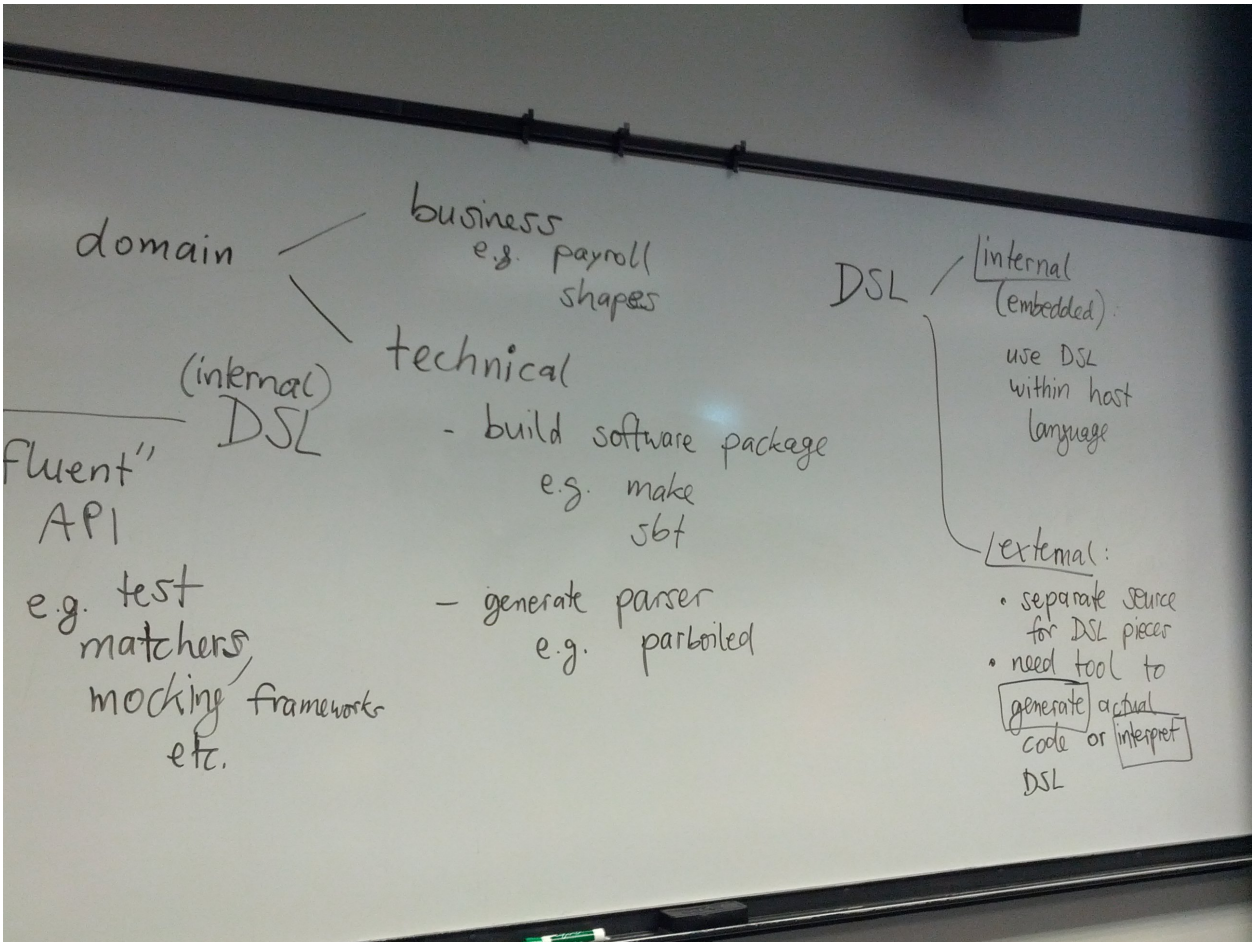
Additional information is available here:

- design space and tradeoffs
- representative examples
- comprehensive list
- presentation slides 10-29
- performance implications/shootout

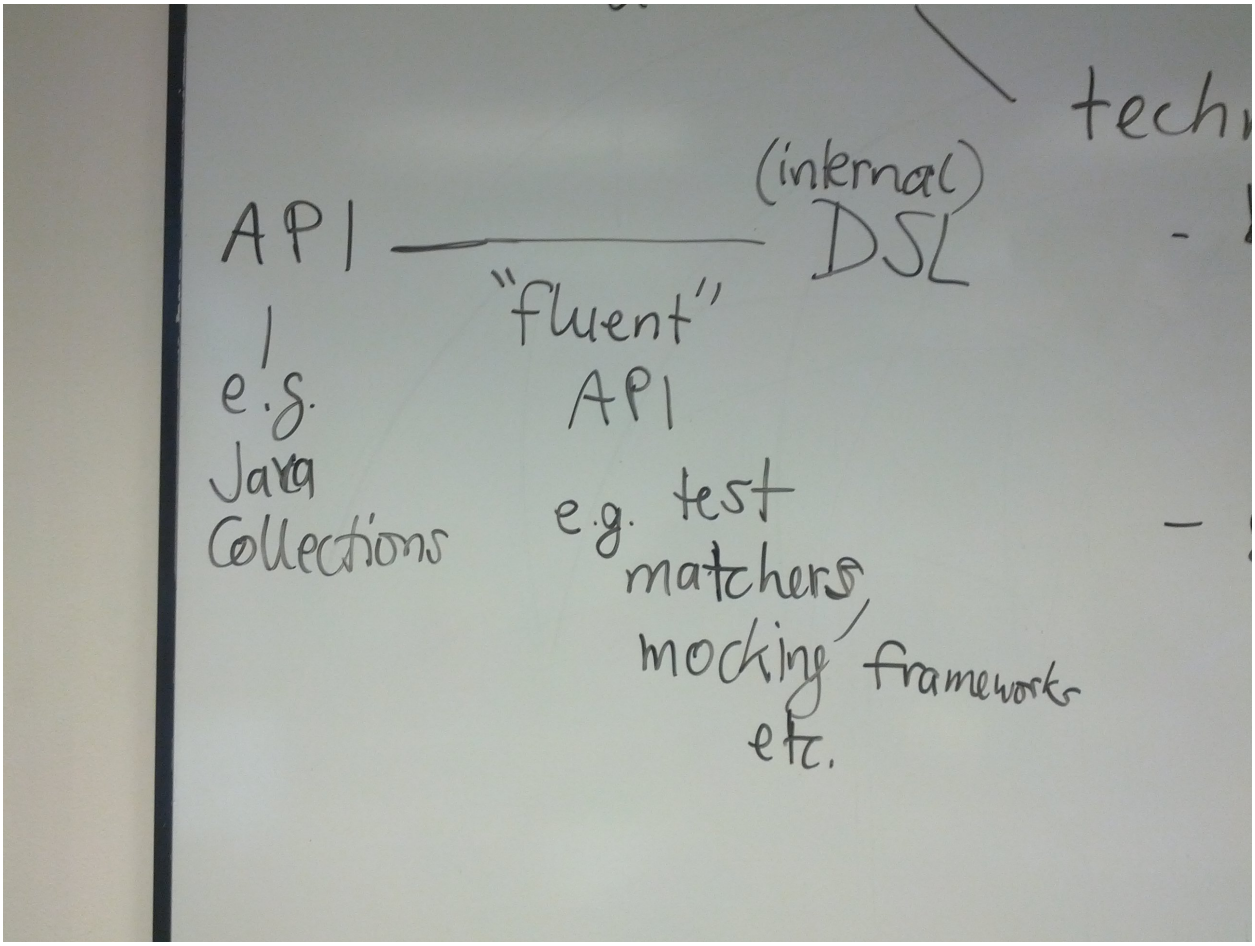
5.7 Domain-specific languages

Domain-specific languages are special-purpose languages for solving problems in particular domains. We identify the following dimensions:

- internal/embedded language versus external language
- business domain versus technical domain



In addition, there is a continuum between APIs and internal DSLs.



Additional information is available here:

- http://c2.com/cgi/wiki?DomainSpecificLanguage
- http://martinfowler.com/bliki/FluentInterface.html
- http://martinfowler.com/bliki/DomainSpecificLanguage.html
- http://martinfowler.com/books/dsl.html
- http://en.wikipedia.org/wiki/Domain-specific_language
- SE Radio episode 182

CHAPTER

SIX

INTRODUCTION TO CONCURRENCY

6.1 Overview

Why and when do we need concurrency?

- When it is a natural fit for the problem domain
  - multiple autonomous behaviors/simulations
  - user interfaces: timed events, background activities
- When the technical solution domain requires it
  - more efficient use of available resources: asynchronous computing
  - graphical user interfaces: queuing of low-level input events
  - multi-core systems
  - network services/distributed systems

Key considerations:

- physical (parallelism) versus logical concurrency
- speedup and when to expect it
- data parallelism versus task parallelism

6.2 Activity terminology and concerns

- process: own memory
- thread: shared memory *and* "thread-local" state
- foreground versus background
- CPU-bound versus IO-bound
  - CPU-bound example
  - IO-bound example
- run-to-completion versus coordination
- progress reporting
- cancelation

6.3 Thread safety

- nondeterminism
  - example
- extent of nondeterminism: see subsection below
- race conditions
  - example
- root cause of thread safety problems

6.3.1 Understanding the extent of nondeterminism

Consider this small example of two concurrent increment operations:

```
/*f1*/ final int local1 = shared;    /*f2*/ final int local2 = shared;
/*s1*/ shared = local1 + 1;          /*s2*/ shared = local2 + 1;
```

When analyzing race conditions, we might be tempted to enumerate the different possible interleavings. While it seems reasonable for this example, this quickly becomes impractical because of the combinatorial explosion for larger number of threads with more steps. (Please see the CDER chapter for more details.)

To appreciate this combinatorial explosion, let's count the possible interleavings for the case of k threads with n steps each. We recall the binomial coefficient i choose j defined as

$$\binom{i}{j} = \frac{i!}{j!(i-j)!} \text{ for } 0 \leq j \leq i$$

In our case, there are kn steps, of which the first thread chooses n; there are $\binom{kn}{n}$ possibilities for this. This leaves $(k-1)n$ steps, of which the second thread chooses n, and so on. At the end, there are n steps left, which are the only choice for the last thread. The total number of choices is the product of choices for each thread:

$$\binom{kn}{n}\binom{(k-1)n}{n}\cdots\binom{2n}{n}\binom{n}{n} = \frac{(kn)!}{n!(kn-n)!}\frac{((k-1)n)!}{n!((k-1)n-n)!}\cdots\frac{(2n)!}{n!(2n-n)!}\frac{(n)!}{n!(n-n)!}$$

Here the second factor in each denominator cancels out against the numerator of the next top-level factor and the second factor in the last denominator is 1, leaving

$$\frac{(kn)!}{n!^k}$$

As the number of threads and/or their number of steps grow beyond two, the number interleavings gets very large.

n/k	$k=2$	$k=3$	$k=4$
$n=2$	6	90	2520
$n=3$	20	1680	369600
$n=4$	70	34650	63063000

Therefore, we cannot attempt to comprehend, let alone enumerate, all possible interleavings. Instead, we need to think in terms of constraints, e.g., f1 always happens before s1, and f2 always happens before s2.

Once we make each thread atomic, however, the number of interleavings shrinks dramatically to $k!$.

6.4 Dealing with shared state

- mutual exclusion/locking
- confinement
- immutability
- case study: GUIs and the single-threaded rule

6.5 (Conflicting) design forces

- correctness/(thread-)safety
- liveness/deadlock
  - dining philosophers example <- *run on Firefox or Safari with site added to Java security settings*
- fairness/starvation
- performance
  - throughput
  - latency
  - jitter

6.6 Specific concurrency mechanisms

Language constructs, patterns, building blocks:

- threads (familiar from 313/413)
- monitors: synchronized/locks, wait/notify
- fully synchronized object (pattern/building blocks)
- Android (also familiar from 313/413)
  - AsyncTask
  - ThreadPoolExecutor
- java.util.concurrent
  - atomic variables
  - thread-safe collections
  - FIFO locks
  - ...
- Scala parallel collections
- futures and promises intro
- composable futures in Scala/Akka
  - example: concurrent web requests
- actors

- reactive streams including Akka streams
- software transactional memory

6.7 References: concurrent and asynchronous computing

- Läufer and Thiruvathukal, CDER chapter (shared on Slack)
- Goetz et al., JCIP
- Doug Lea, CPJ
- Thiruvathukal and Christopher, HPJPC
- SE Radio episode on concurrency: part 1
- SE Radio episode on concurrency: part 2
- SE Radio episode on concurrency: part 3
- SE Radio episode on concurrency: part 4
- futures and promises overview
- RxJava/RxScala
- asynchronous programming video
- reactive/asynchronous programming with RxJava/RxScala video

CHAPTER

SEVEN

SUMMARY

summary/recap goes here

CHAPTER

EIGHT

APPENDIX: PROGRAMMING LANGUAGE DESIGN PRINCIPLES

This is the list of programming language design principles from Bruce MacLennan's excellent Principles of Programming Languages (2nd edition, 1986, ISBN 0-03-005163-0, found on Oliver Wyman's blog.

- **Abstraction**: Avoid requiring something to be stated more than once; factor out the recurring pattern.
- **Automation**: Automate mechanical, tedious, or error-prone activities.
- **Defense in Depth**: Have a series of defences so that if an error isn't caught by one, it will probably be caught by another.
- **Information Hiding**: The language should permit modules designed so that (1) the user has all of the information needed to use the module correctly, and nothing more; and (2) the implementor has all of the information needed to implement the module correctly, and nothing more.
- **Labeling**: Avoid arbitrary sequences more than a few items long. Do not require the user to know the absolute position of an item in a list. Instead, associate a meaningful label with each item and allow the items to occur in any order.
- **Localized Cost**: Users should only pay for what they use; avoid distributed costs.
- **Manifest Interface**: All interfaces should be apparent (manifest) in the syntax.
- **Orthogonality**: Independent functions should be controlled by independent mechanisms.
- **Portability**: Avoid features or facilities that are dependent on a particular machine or a small class of machines.
- **Preservation of Information**: The language should allow the representation of information that the user might know and that the compiler might need.
- **Regularity**: Regular rules, without exceptions, are easier to learn, use, describe, and implement.
- **Security**: No program that violates the definition of the language, or its own intended structure, should escape detection.
- **Simplicity**: A language should be as simple as possible. There should be a minimum number of concepts, with simple rules for their combination.
- **Structure**: The static structure of the program should correspond in a simple way to the dynamic structure of the corresponding computations.
- **Syntactic Consistency**: Similar things should look similar; different things different.
- **Zero-One-Infinity**: The only reasonable numbers are zero, one, and infinity.

CHAPTER

NINE

APPENDIX: RESOURCES

Todo

consider moving most of these into the corresponding chapters as "further reading"

9.1 Key resources

- Code examples on GitHub
- Instructor's videos on YouTube
- External playlist on YouTube
- External references/bookmarks on Diigo
- General resources
- Coding guidelines
- Library of research resources (LRR)
- Loyola library's CS subject guide
- Some professional meetups around town

9.2 Code examples

The code examples for this course are at https://github.com/lucproglangcourse. (The non-Scala ones are much less likely to change.)

The recommended process for exploring an example is to fork it into your own GitHub account. Please create issues and/or submit pull requests with improvements or corrections.

It is important for you to receive notifications of changes to the key examples.

- These will automatically appear on this Slack channel, which you can join.
- If you have a GitHub account, you can instead receive notifications within GitHub by watching and/or starring the repositories of interest directly.

9.3 Required text

- **Programming in Scala, Third Edition** *By: Martin Odersky; Lex Spoon; Bill Venners* Publisher: Artima Press Pub. Date: April 28, 2016 ISBN-10: 0-9815316-8-7 ISBN-13: 978-0-9815316-8-7 available free on Safari

9.4 Recommended/supplementary texts

- **Scala Test-Driven Development** *By: Gaurav Sood* Publisher: Packt Publishing Pub. Date: October 27, 2016 Web ISBN-13: 978-1-78646-302-9 Print ISBN-13: 978-1-78646-467-5 available free on Safari
- **Basics of Compiler Design** *By: Torben Mogensen* available online
- **Learning Concurrent Programming in Scala** *By: Aleksandar Prokopec* Publisher: Packt Publishing Pub. Date: November 28, 2014 Print ISBN-13: 978-1-78328-141-1 Web ISBN-13: 978-1-78328-142-8 available free on Safari
- **Seven Concurrency Models in Seven Weeks** *By: Paul Butcher* Publisher: Pragmatic Bookshelf Pub. Date: June 30, 2014 Print ISBN-13: 978-1-937785-65-9 available free on Safari

9.5 Additional texts on Scala

- **Learning Scala** *By: Jason Swartz* Publisher: O'Reilly Media, Inc. Pub. Date: December 24, 2014 Print ISBN-13: 978-1-4493-6793-0 available free on Safari
- **A Concise Introduction to Scala** *By: David Matuszek* available online
- **Scala for the Impatient** *By: Cay S. Horstmann* Publisher: Addison-Wesley Professional Pub. Date: March 06, 2012 Print ISBN-10: 0-321-77409-4 Print ISBN-13: 978-0-321-77409-5 Web ISBN-10: 0-13-276177-7 Web ISBN-13: 978-0-13-276177-2 first half available free from Typesafe
- **Scala for Java Developers** *By: Thomas Alexandre* Publisher: Packt Publishing Pub. Date: April 25, 2014 Print ISBN-13: 978-1-78328-363-7 Web ISBN-13: 978-1-78328-364-4 available free on Safari
- **Functional Programming in Scala** (advanced) *By: Paul Chiusano and Rúnar Bjarnason* Foreword by Martin Odersky Publisher: Manning Publications Pub. Date: August 2014 Print ISBN: 9781617290657 available free on Safari

9.6 Online Scala resources

- SIGCSE Scala workshop
- The Neophyte's Guide to Scala
- Twitter Scala School
- Scala resources from an Applied Text Analysis course
- Scala documentation site
- A Scala Tutorial for Java programmers
- Scala API documentation
- Scala style guide
- Scala parser combinator tutorial

- parboiled2 parser generator
- scalaz API documentation (advanced)
- scalaz cheat sheet (advanced)

9.7 Resources on program representation and interpretation

- Compiler design tutorial
- Abstract syntax tree
- Regular expressions
- Context-free grammars (CFGs)
- Extended BNF
- Parser generators
- Parser combinators
- Parsing expression grammars (PEGs)
- Interpreters

9.8 Other resources

- Bruce Maclennan's programming language design principles
- Seven languages in seven weeks
- 10 programming languages that could shake up IT
- 7 programming languages on the rise
- *Growing a Language* by Guy Steele: video PDF
- *Origami Programming* by Jeremy Gibbons: PDF
- *On Understanding Types, Data Abstraction, and Polymorphism* by Cardelli and Wegner PDF
- Learn You a Haskell for Great Good!
- **Masterminds of Programming, 1st Edition** *By: Federico Biancuzzi; Shane Warden* Publisher: O'Reilly Media, Inc. Pub. Date: March 27, 2009 Print ISBN-13: 978-0-596-51517-1 available free on EBL
- **Programming Language Pragmatics** (3rd ed.) *By: Michael L. Scott* ISBN: 978-0-12-374514-9 Pub date: Mar 23, 2009 Elsevier Science & Technology companion site
- **Functional Thinking** *By: Neal Ford* Publisher: O'Reilly Media, Inc. Pub. Date: July 10, 2014 Print ISBN-13: 978-1-4493-6551-6 available free on Safari
- **Real World Haskell** *By: Bryan O'Sullivan, Don Stewart, and John Goerzen* Publisher: O'Reilly Media. Pub. Date: November 2008 Print ISBN-10: 0596514980 Print ISBN-13: 978-0596514983 available online for free
- **Haskell: The Craft of Functional Programming (3rd Edition)** *By: Simon Thompson* Publisher Addison-Wesley Professional Pub. Date: 2012 Print ISBN-13: 9780201882957 online companion site

CHAPTER

TEN

APPENDIX: COURSE SOFTWARE

You can choose either or both of the following options.

10.1 Cloud-based development environment

This option is recommended if you have any problems with your local setup or if you prefer a basic cloud-based setup.

10.1.1 Pros

- consistent, cloud-based environment

10.1.2 Cons

- requires good network connection
- lacks code completion

10.1.3 Setup

- Use your GitHub, Visual Studio Team Services, or Google+ account to log into Codenvy.
- If you do not have any of these accounts, create a GitHub account. Please see below for more information on GitHub.
- Create a new workspace with the following settings:
  - New from blank ...
  - Create new workspace from stack > BLANK
  - Name: lucproglangcourse
  - RAM: 3GB
- Wait for the workspace to become available. Now you have a persistent, cloud-based, standard Ubuntu/Linux command-line environment that you can work in and come back to from any computer.
- In the resulting Linux terminal, run this gist to install the Scala build tool (sbt):

```
wget https://gist.github.com/klaeufer/c4040de09f51395b96be/raw/install-sbt-ubuntu.
↪sh && sh install-sbt-ubuntu.sh
```

- Now you can check out existing projects using git or create a new one like so:

```
$ mkdir -p hello/src/main/scala
$ cd hello
$ cat > src/main/scala/Hello.scala
object Hello extends App {
  println("hello")
}
^D
$ sbt run
...
hello
```

10.2 Locally installed development environment

This option will give you an advanced development environment with code completion, type info, etc.

10.2.1 Pros

- provides powerful capabilities, including code completion
- does not require network connection

10.2.2 Cons

- might be slow on older machines
- need to maintain on each machine you use

10.2.3 Required components

- Java 8 and/or newer JDK
- Git distributed version control system (usually preinstalled on Mac OS and Linux)
  - recommended installation option on Windows: *Use Git and optional Unix tools from the Windows Command Prompt*
- sbt Scala build tool
- IntelliJ IDEA CE integrated development environment
  - check specific prerequisite details for your platform
  - for the following steps, make sure you have no projects open and are looking at the welcome window as in the attached screenshot
  - *Scala plugin installation:* IntelliJ IDEA > Configure (bottom right) > Plugins > Browse repositories > find and right-click Scala > download and install > close repository browser > OK to restart IDEA
  - *JDK configuration*: IntelliJ IDEA > Configure > Project Defaults > Project Structure > Platform Settings > SDKs > + > JDK > navigate to the installation directory of your most recent JDK > OK

10.2.4 Optional for Windows users

- TortoiseGit integration of Git with Windows Explorer
- SourceTree Git client

10.2.5 Optional for Mac users

- SourceTree Git client

10.2.6 Optional for all IntelliJ users

- These are useful Android Studio/Intellij IDEA plugins. (Installation procedure is the same as for the Scala plugin.)
  - Key Promoter (helps you learn keyboard shortcuts)
  - MultiMarkdown

10.3 sbt optional plugins

This section applies to all uses of sbt, whether cloud-based or on the local command line.

These are useful additional sbt plugins. You can install them per project or globally. Some of the example projects already come with one or more of these plugins.

- sbt-scoverage: uses Scoverage to produce a test code coverage report
- sbt-updates: checks central repos for dependency updates
- sbt-scalariform: automatic source code formatting using Scalariform (see also this Gist)
- sbt-native-packager: creates a script for running your app outside sbt
- ls-sbt: browse available libraries on GitHub using ls.implicit.ly
- sbt-dependency-graph: creates a visual representation of library dependency tree
- cpd4sbt: copy/paste detection for Scala
- scalastyle: static code checker for Scala
- wart remover: code linting tool (another static checker)
- sbt-stats: simple, extensible source code statistics/metrics

10.4 Bitbucket

Bitbucket is a provider of hosted Git repositories. We use it for project submission and collaboration on team projects.

- Create an account on if you do not yet have one.
- To get upgraded to the top plan for free, associate your account with your official LUC email address.
- Use this excellent documentation to set up public key authentication for your environment. This will allow you to use Git on the command line without having to enter your password.

10.5 GitHub (optional/recommended)

GitHub is another provider of hosted Git repositories, which emphasizes community and collaboration. For this reason, we use it to host our course examples.

- Create a GitHub account if you don't already have one.
- Find and follow a few practitioners you respect. For example, I follow these developers. You'll probably recognize a number of them.
- Review these notes to understand the community-based development process.
- For credit toward class participation, create some meaningful GitHub issues and/or GitHub pull requests for one or more of our course examples. (Make sure to navigate to the original repos as these forks do not have their own issue trackers). These can be functional or nonfunctional enhancements, requests for clarification, etc.
- To enhance your visibility in the professional community, start doing the same for some open-source projects you are interested in.
- Consider getting the GitHub Student Developer Pack, which includes µ account with five private repos.

10.6 Remote participation

This application enables you to grant others remote access to your desktop. This is helpful for diagnosing and overcoming obstacles without having to pair with the instructor in person. The application is free for personal use and does not require registration.

- TeamViewer

This software allows you to participating in class remotely in case of weather emergencies.

- Adobe Connect

CHAPTER

ELEVEN

APPENDIX: COURSE SYLLABUS

This is the official course syllabus for the current section(s) of this course. Under construction for spring 2018.

11.1 Course: COMP 372/471 Programming Languages

- Prerequisite: Comp 313
- Official course description: Comp 372 | Comp 471

11.2 Section: 001 Spring 2018

- *Required text:* **Scala for the Impatient, Second Edition** *By: Cay S. Horstmann* Publisher: Addison-Wesley Professional Pub. Date: December 15, 2016 Web ISBN-13: 978-0-13-454062-7 Print ISBN-13: 978-0-13-454056-6 available free on Safari

11.3 Section: 001 Spring 2017

- *Instructor:* Konstantin Läufer
- *Class time and location (Spring 2017)*: Tue 14:45-17:15, Mundelein Center 607, LSC
- *Office hour:* Tue 17:15-18:00 in Mundelein Center 607, LSC, and by appointment
- *Required text:* **Programming in Scala, Third Edition** *By: Martin Odersky; Lex Spoon; Bill Venners* Publisher: Artima Press Pub. Date: April 28, 2016 ISBN-10: 0-9815316-8-7 ISBN-13: 978-0-9815316-8-7 available free on Safari
- Grading (tentative):
  - 45% quizzes & exams
  - 50% projects & presentations *(Percentage effort on each group project will be measured by an end-of-term questionnaire. Group project grades and/or final course grades may be adjusted to account for significant discrepancies in effort among group members.)*
  - 5% participation (in-class and online, including announcements of and reports from relevant professional events, GitHub issues and PRs for course examples, etc.)
- Ground rules
- Sakai site for this section (gradebook)

- Slack team *(mandatory subscription)*

11.4 Detailed Course Outline

- functional programming (6 weeks)
  - defining algebraic data types
    - scalars: enumerations
    - sublinear structures: numbers, option
    - linear structures: lists, maps
    - nonlinear structures: trees
  - implementing behaviors on algebraic data types
    - pattern matching
    - recursion
  - higher-order functions
  - predefined types and their behaviors
  - recursion patterns
  - higher-kinded types
- programming language representation and interpretation/execution (5 weeks)
  - language design space
  - scanners (lexical analyzers)
  - parsers (syntax analyzers)
  - interpreters and compilers
  - domain-specific languages
- concurrent and parallel programming (4 weeks)
  - parallel collections
  - futures and promises
  - progress reporting and cancelation
  - asynchronous programming/reactive extensions (Rx)
  - advanced mechanisms
    - explicit threads
    - actors
    - software-transactional memory
    - task-parallel library
    - functional data structures

11.5 Important Dates

- Tue 7 February: quiz 1
- Tue 7 March: NO CLASS - spring break
- Tue 14 March: test 1
- Tue 4 April: quiz 2
- Tue 18 April: test 2
- Tue 2 May **16:15-18:15**: test 3

11.6 Key Resources

- Code examples on GitHub
- Instructor's videos on YouTube
- External playlist on YouTube
- External references/bookmarks on Diigo
- General resources
- Coding guidelines
- Library of research resources (LRR)
- Loyola library's CS subject guide
- Some professional meetups around town

CHAPTER

TWELVE

APPENDIX: TODO

Todo

expand into a proper section

(The original entry is located in /Users/laufer/Cloud/Dropbox/Home/Work/writing/lucproglangcourse/source/background.rst, line 137.)

Todo

elaborate more on `for` comprehensions and `flatMap`

(The original entry is located in /Users/laufer/Cloud/Dropbox/Home/Work/writing/lucproglangcourse/source/functionalprogramming.rst line 41.)

Todo

add more recent idioms including successive `Try` and `for` with blocks for embedding stateful steps such as logging

(The original entry is located in /Users/laufer/Cloud/Dropbox/Home/Work/writing/lucproglangcourse/source/functionalprogramming.rst line 98.)

Todo

Practical applications

(The original entry is located in /Users/laufer/Cloud/Dropbox/Home/Work/writing/lucproglangcourse/source/functionalprogramming.rst line 275.)

Todo

put chapter-level references here

(The original entry is located in /Users/laufer/Cloud/Dropbox/Home/Work/writing/lucproglangcourse/source/functionalprogramming.rst line 451.)

Todo

examples below after discussing the next topic

(The original entry is located in /Users/laufer/Cloud/Dropbox/Home/Work/writing/lucproglangcourse/source/imperative.rst, line 277.)

Todo

add sample projects, activities, tests, and (tiered) master list for presentations

(The original entry is located in /Users/laufer/Cloud/Dropbox/Home/Work/writing/lucproglangcourse/source/index.rst, line 53.)

Todo

add stuff from https://trello.com/c/kAMGZ4uf/28-add-in-next-iteration

(The original entry is located in /Users/laufer/Cloud/Dropbox/Home/Work/writing/lucproglangcourse/source/index.rst, line 55.)

Todo

incorporate visual presentation here

(The original entry is located in /Users/laufer/Cloud/Dropbox/Home/Work/writing/lucproglangcourse/source/overview.rst, line 41.)

Todo

map Bloom levels to course outline

(The original entry is located in /Users/laufer/Cloud/Dropbox/Home/Work/writing/lucproglangcourse/source/overview.rst, line 61.)

Todo

consider moving most of these into the corresponding chapters as "further reading"

(The original entry is located in /Users/laufer/Cloud/Dropbox/Home/Work/writing/lucproglangcourse/source/resources.rst, line 5.)

- genindex
- search

Todo

add sample projects, activities, tests, and (tiered) master list for presentations

Todo

add stuff from https://trello.com/c/kAMGZ4uf/28-add-in-next-iteration